Import Libraries

In [1]:
```
import pandas as pd
import numpy as np
import warnings
warnings.filterwarnings("ignore")

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import roc_auc_score, roc_curve

import statsmodels.api as sm
```

Load Data

In [2]:
```
df=pd.read_csv(r'C:\Users\anupa\Desktop\Python Learning\Credit Risk Model\loan.csv', low_memory=False)
print(df.shape)
```

```
(2260668, 145)
```

Target Variable

In [3]:
```
df = df[df['loan_status'].isin([
    'Fully Paid','Charged Off','Default'
])]

df['target'] = np.where(
    df['loan_status'].isin(['Charged Off','Default']),1,0
)
```

Remove Data Leakage Variables

In [4]:
```
leakage_cols = [

'id','member_id','url','zip_code',

'out_prncp','out_prncp_inv',

'total_pymnt','total_pymnt_inv',

'total_rec_prncp','total_rec_int',

'total_rec_late_fee','recoveries',

'collection_recovery_fee',

'last_pymnt_d','last_pymnt_amnt',

'collections_12_mths_ex_med',

'policy_code','acc_now_delinq',

'delinq_amnt','last_credit_pull_d',

'loan_status'
]

df = df.drop(columns=[c for c in leakage_cols if c in df.columns])
```

Remove High Missing Columns

In [5]:
```
missing_ratio = df.isnull().mean()

drop_cols = missing_ratio[missing_ratio>0.7].index

df = df.drop(columns=drop_cols)

print("Remaining variables:",df.shape[1])
```

```
Remaining variables: 86
```

Train / Validation Split

In [6]:
```
X = df.drop('target',axis=1)
y = df['target']

X_train,X_val,y_train,y_val = train_test_split(

    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

train_df = pd.concat([X_train,y_train],axis=1)
val_df = pd.concat([X_val,y_val],axis=1)

print(train_df.shape,val_df.shape)
```

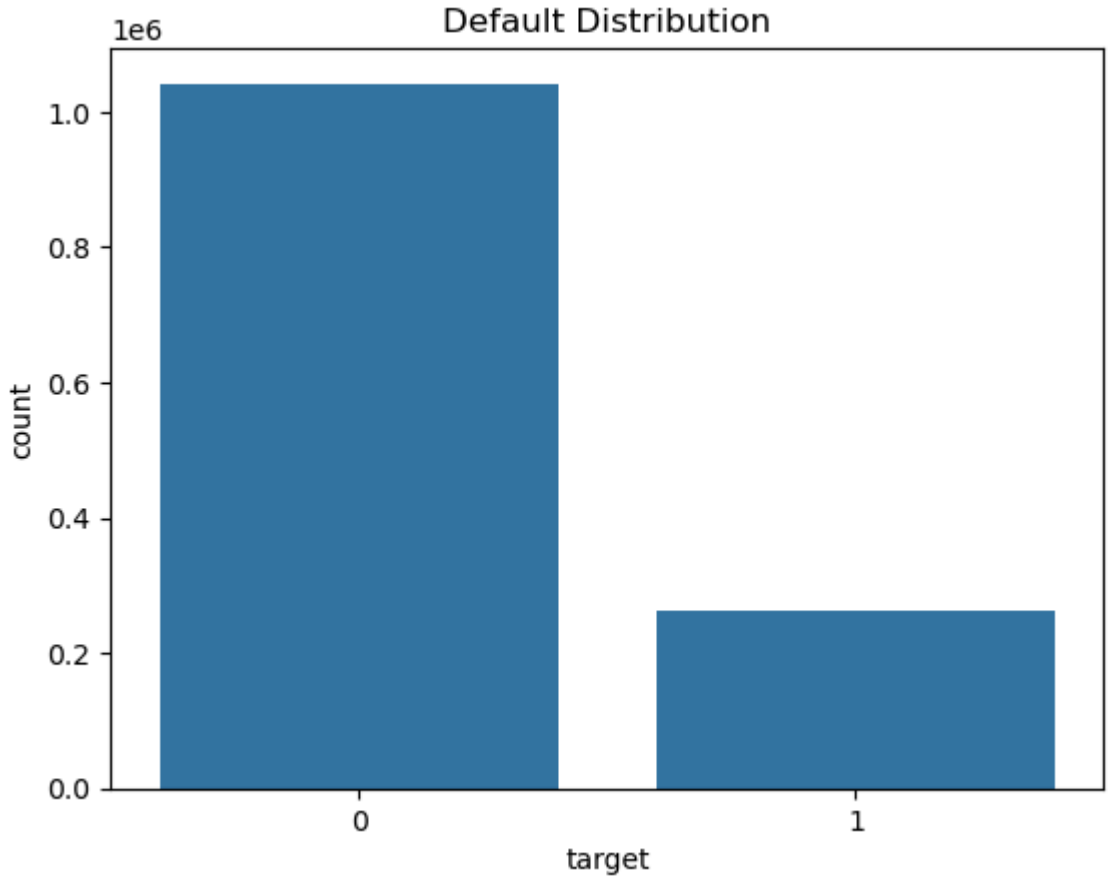
```
(912546, 86) (391092, 86)
```

Missing Value Treatment

In [7]:
```
num_cols = train_df.select_dtypes(include=['int64','float64']).columns.tolist()
num_cols.remove('target')

cat_cols = train_df.select_dtypes(include=['object']).columns.tolist()

for col in num_cols:
    train_df[col] = train_df[col].fillna(train_df[col].median())
    val_df[col] = val_df[col].fillna(train_df[col].median())

for col in cat_cols:
    train_df[col] = train_df[col].fillna("Missing")
    val_df[col] = val_df[col].fillna("Missing")
```

Target Distribution

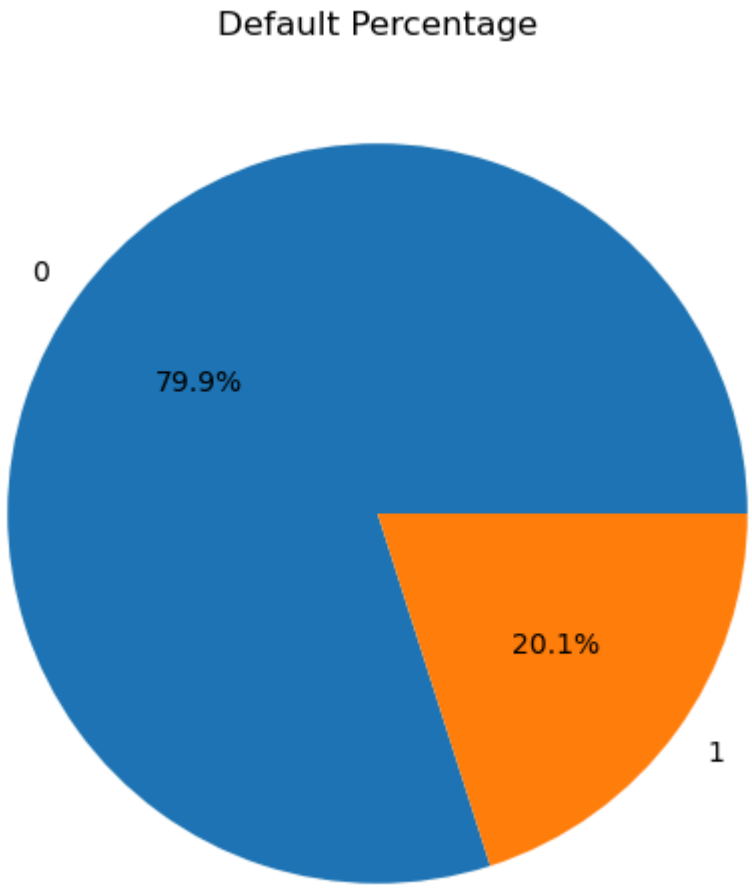
In [8]:
```
sns.countplot(x='target', data=df)

plt.title("Default Distribution")

plt.show()
```



Default Percentage Pie Chart

In [9]:
```
df['target'].value_counts().plot.pie(
    autopct='%1.1f%%',
    figsize=(6,6),
```

```
    title="Default Percentage"
)
plt.ylabel("")
plt.show()
```



Random Forest Feature Selection (TRAIN ONLY)

In [10]:
```
rf_train = train_df.copy()
```

```
# simple encoding for RF
for col in cat_cols:
    rf_train[col] = rf_train[col].astype('category').cat.codes

X_rf = rf_train.drop('target',axis=1)
y_rf = rf_train['target']

rf = RandomForestClassifier(

    n_estimators=300,
    max_depth=10,
    min_samples_leaf=50,
    n_jobs=-1,
    random_state=42
)

rf.fit(X_rf,y_rf)

importance = pd.DataFrame({

"variable":X_rf.columns,
"importance":rf.feature_importances_

}).sort_values(by='importance',ascending=False)

top30 = importance.head(30)['variable'].tolist()

print("Top variables:",top30)
```

```
Top variables: ['debt_settlement_flag', 'sub_grade', 'grade', 'int_rate', 'term', 'dti', 'tot_hi_cred_lim', 'avg_cur_bal', 'acc
_open_past_24mths', 'verification_status', 'mort_acc', 'funded_amnt_inv', 'loan_amnt', 'tot_cur_bal', 'funded_amnt', 'bc_open_t
o_buy', 'all_util', 'installment', 'open_rv_24m', 'home_ownership', 'annual_inc', 'total_bc_limit', 'num_tl_op_past_12m', 'max_
bal_bc', 'mo_sin_old_rev_tl_op', 'num_actv_rev_tl', 'emp_length', 'num_rev_tl_bal_gt_0', 'total_rev_hi_lim', 'revol_util']
```

Dataset with Selected Variables

In [11]:
```
train_df = train_df[top30 + ['target']]
val_df = val_df[top30 + ['target']]

num_cols = train_df.select_dtypes(include=['int64','float64']).columns.tolist()
```

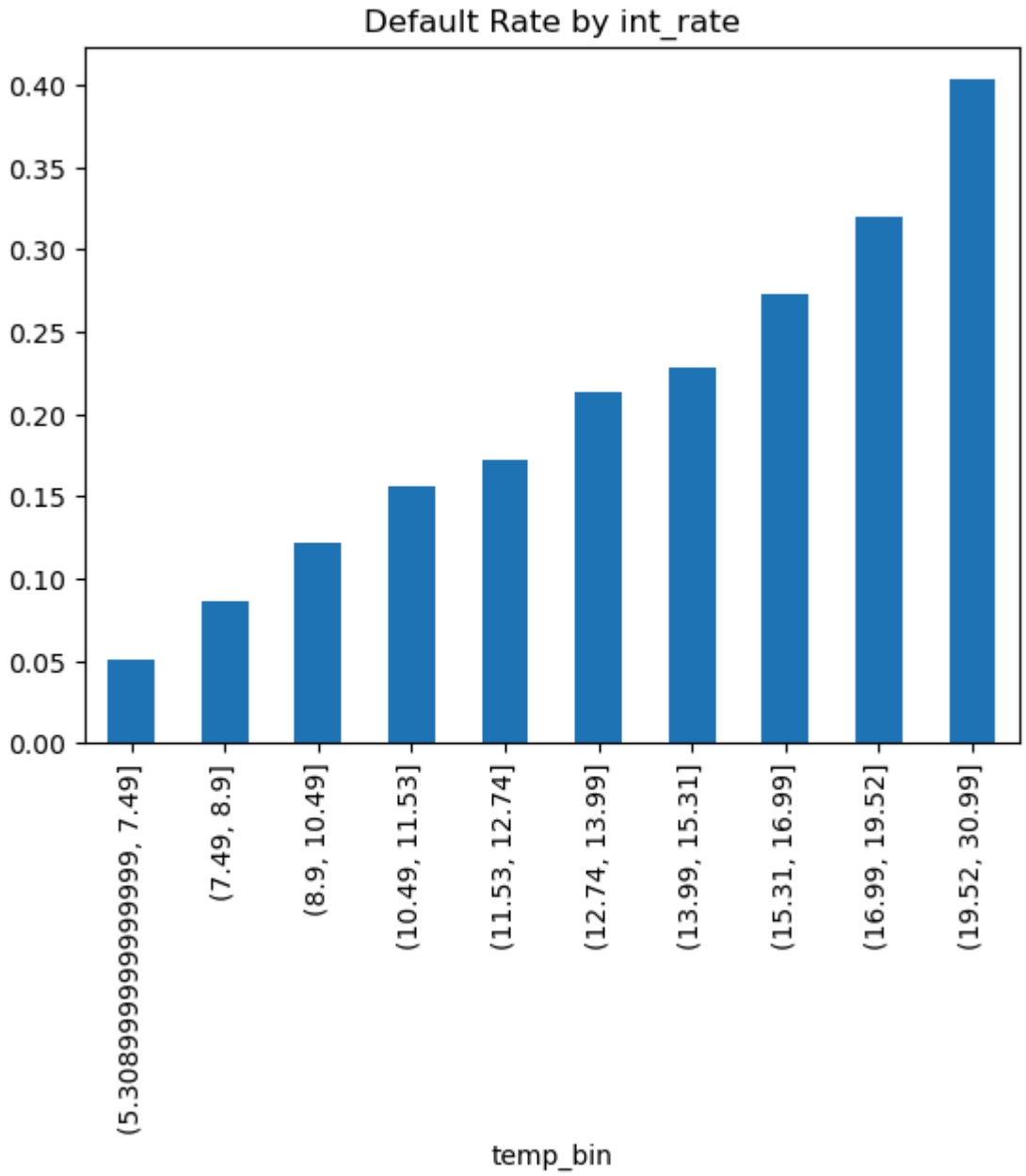
```
num_cols.remove('target')

cat_cols = train_df.select_dtypes(include=['object']).columns.tolist()
```

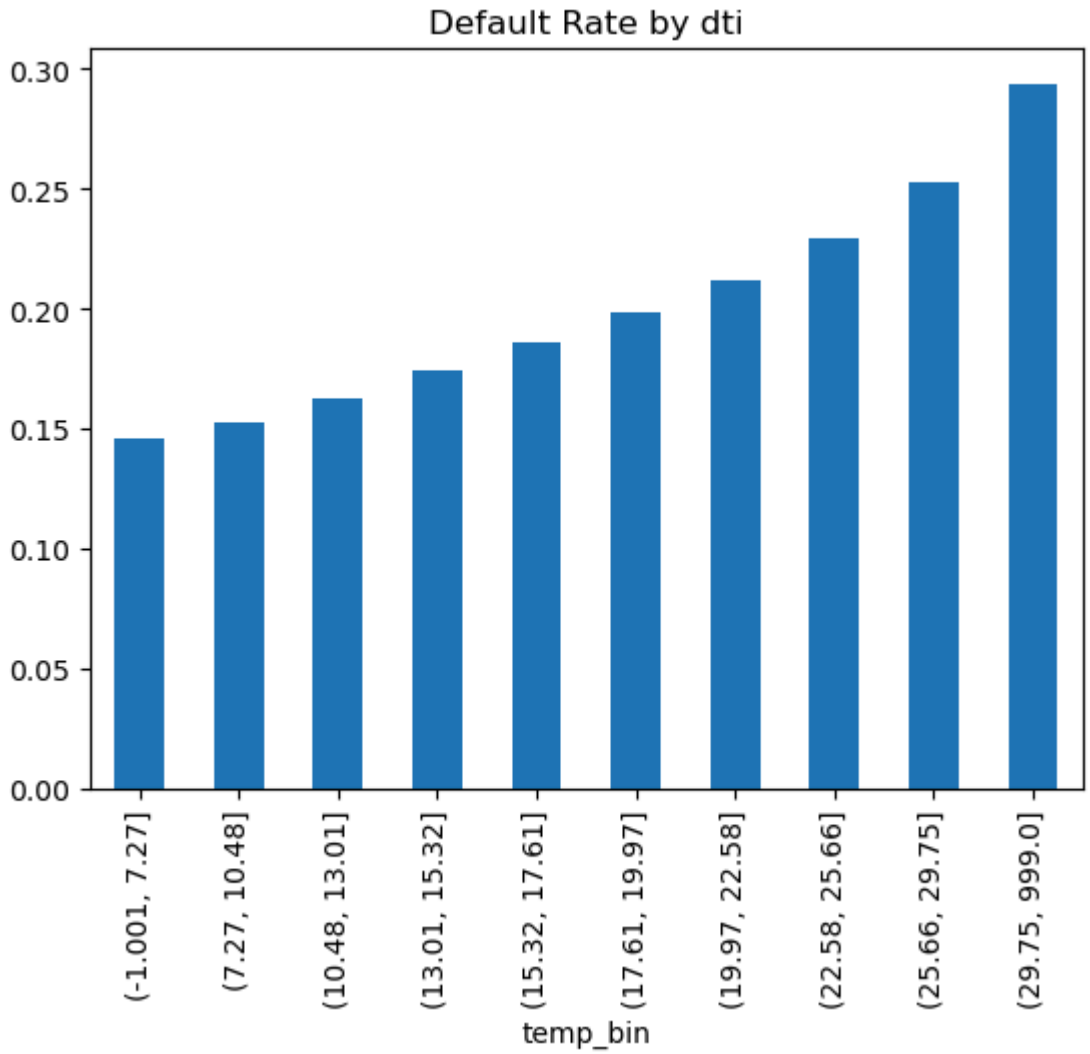
Correlation Removal

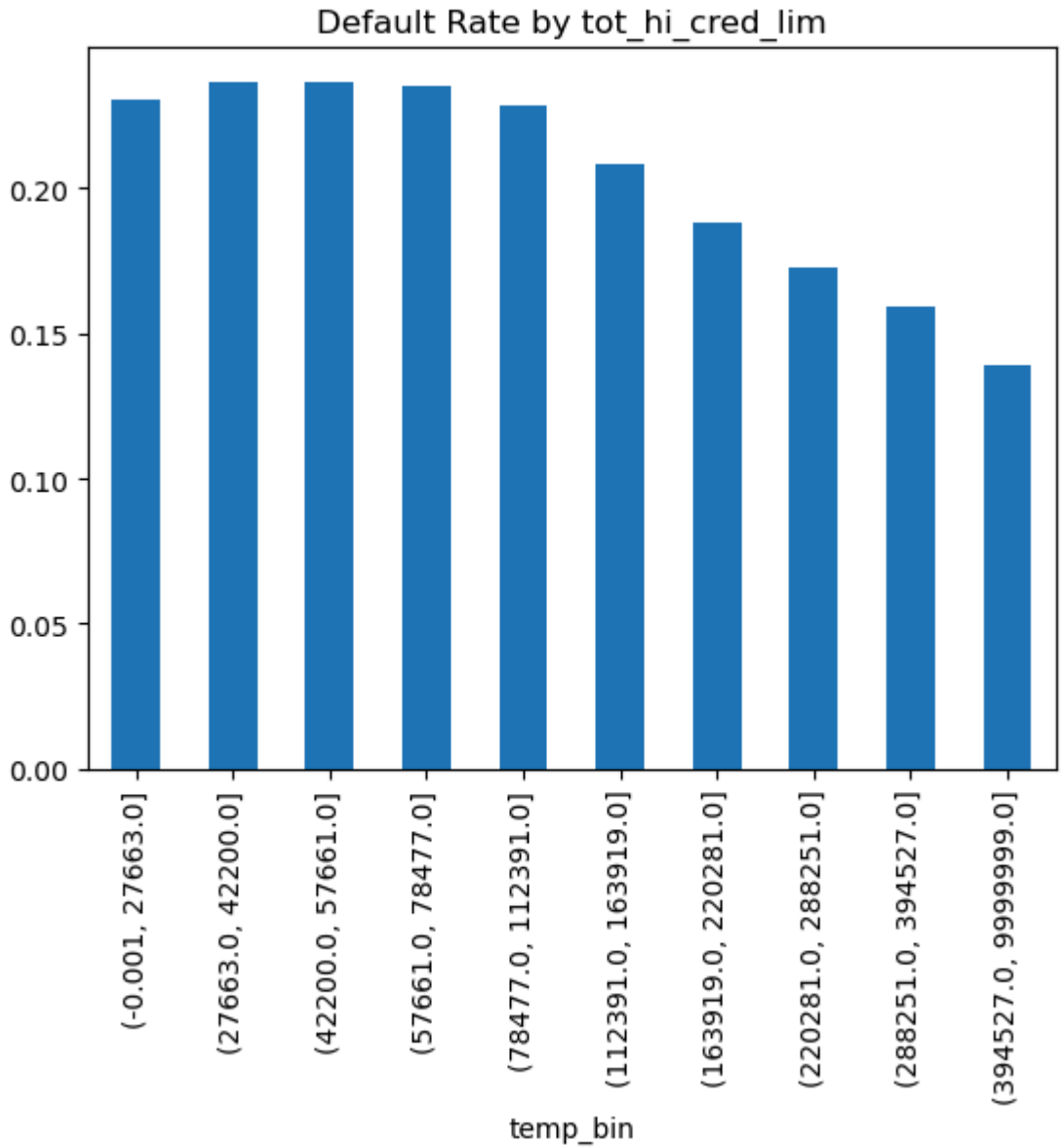
In [12]:
```
corr = train_df[num_cols].corr()

upper = corr.where(np.triu(np.ones(corr.shape),k=1).astype(bool))

high_corr = [

c for c in upper.columns
if any(upper[c]>0.7)

]

train_df = train_df.drop(columns=high_corr)
val_df = val_df.drop(columns=high_corr)

num_cols = [c for c in num_cols if c not in high_corr]
```

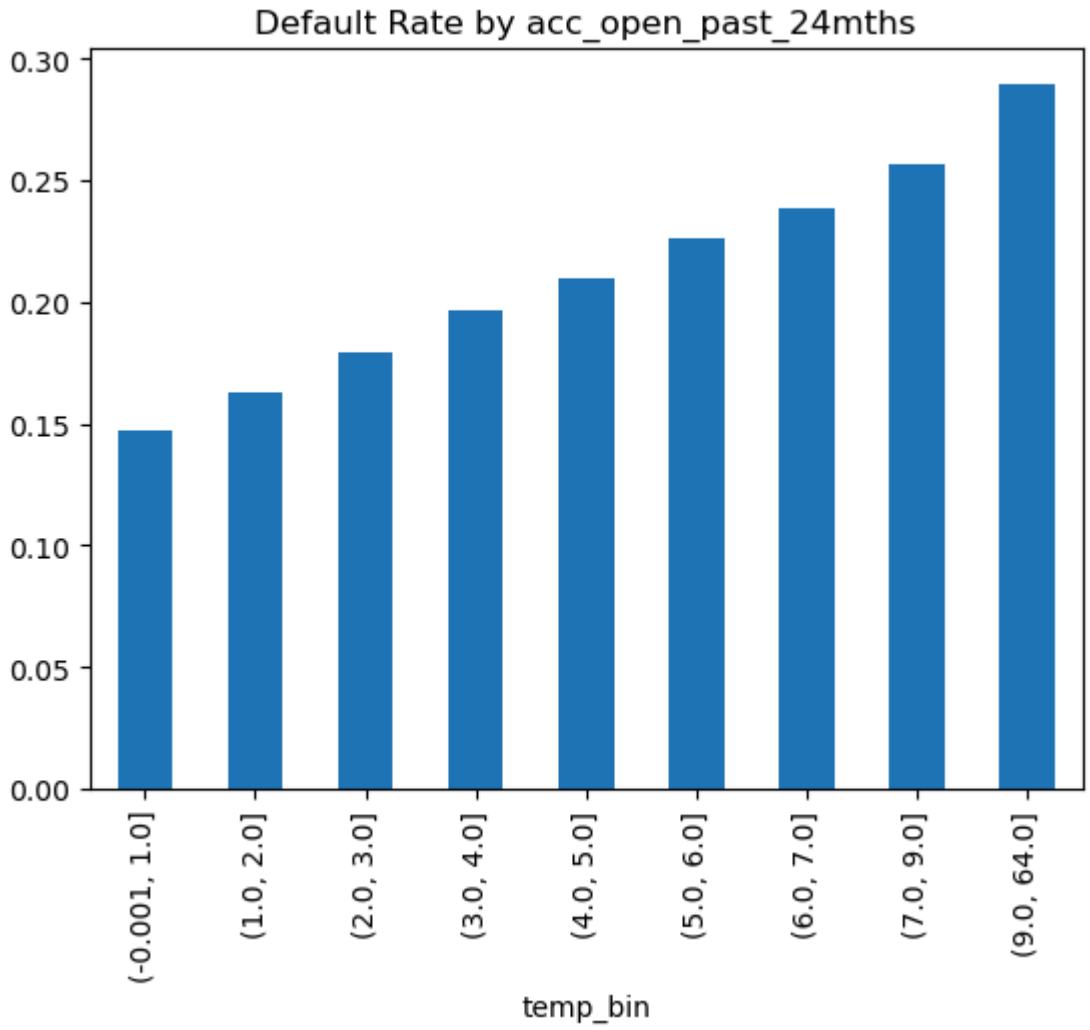
Default Rate by Numerical Variable

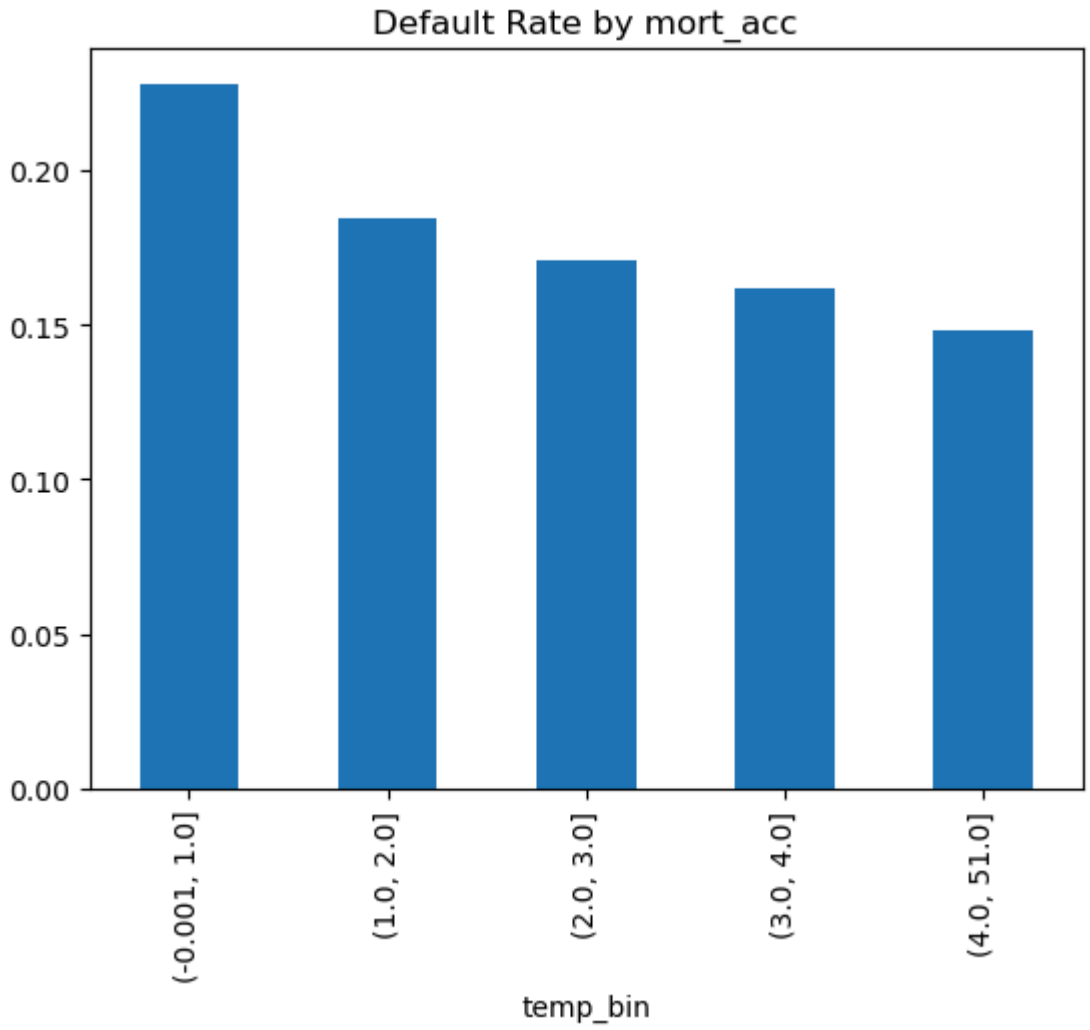
In [13]:
```
for col in num_cols:

    df['temp_bin'] = pd.qcut(df[col],10,duplicates='drop')

    default_rate = df.groupby('temp_bin')['target'].mean()

    default_rate.plot(kind='bar')

    plt.title(f"Default Rate by {col}")

    plt.show()
```

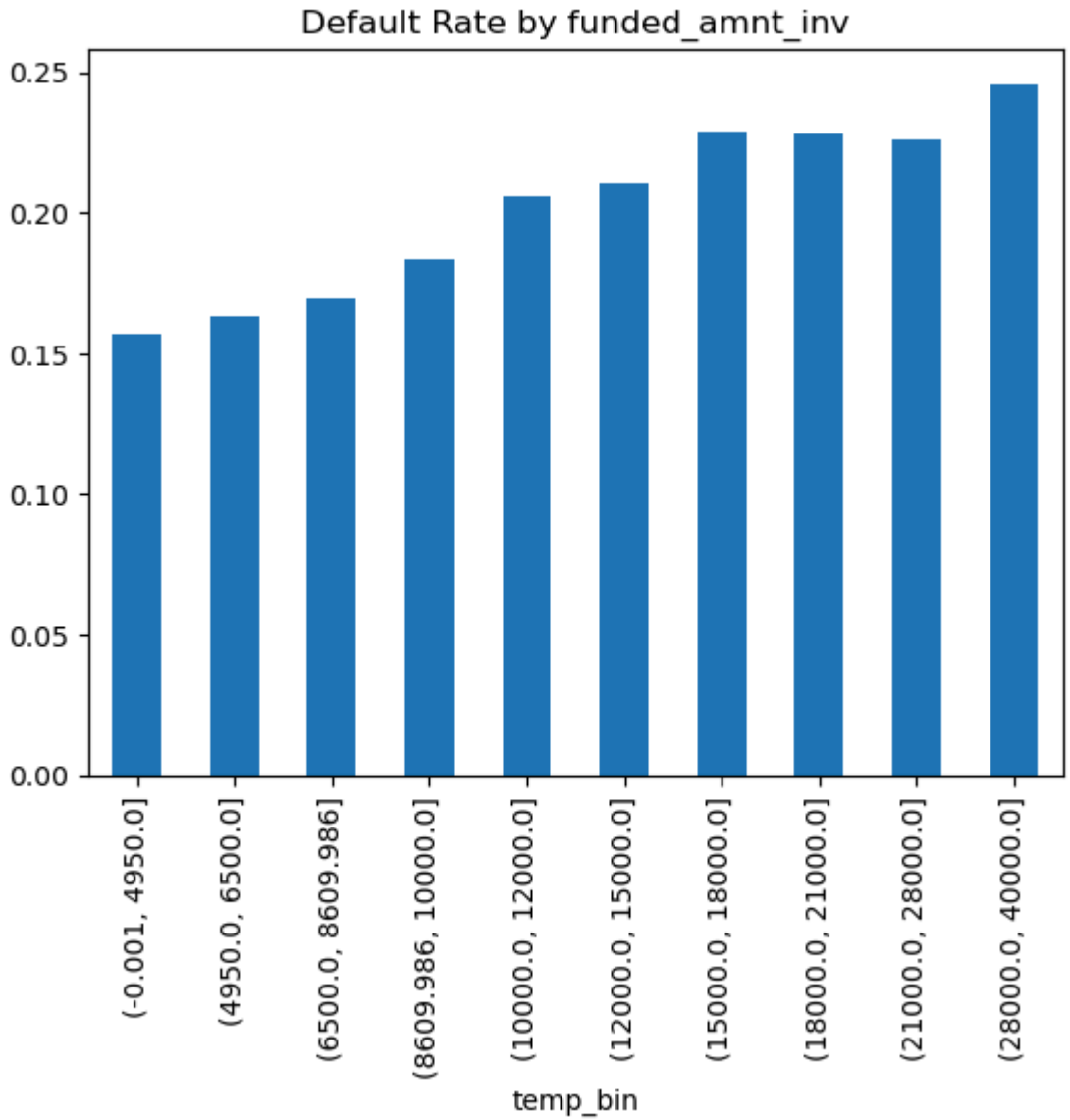

Default Rate by int_rate
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
(5.308999999999999, 7.49]
(7.49, 8.9]
(8.9, 10.49]
(10.49, 11.53]
(11.53, 12.74]
(12.74, 13.99]
(13.99, 15.31]
(15.31, 16.99]
(16.99, 19.52]
(19.52, 30.99]
temp_bin

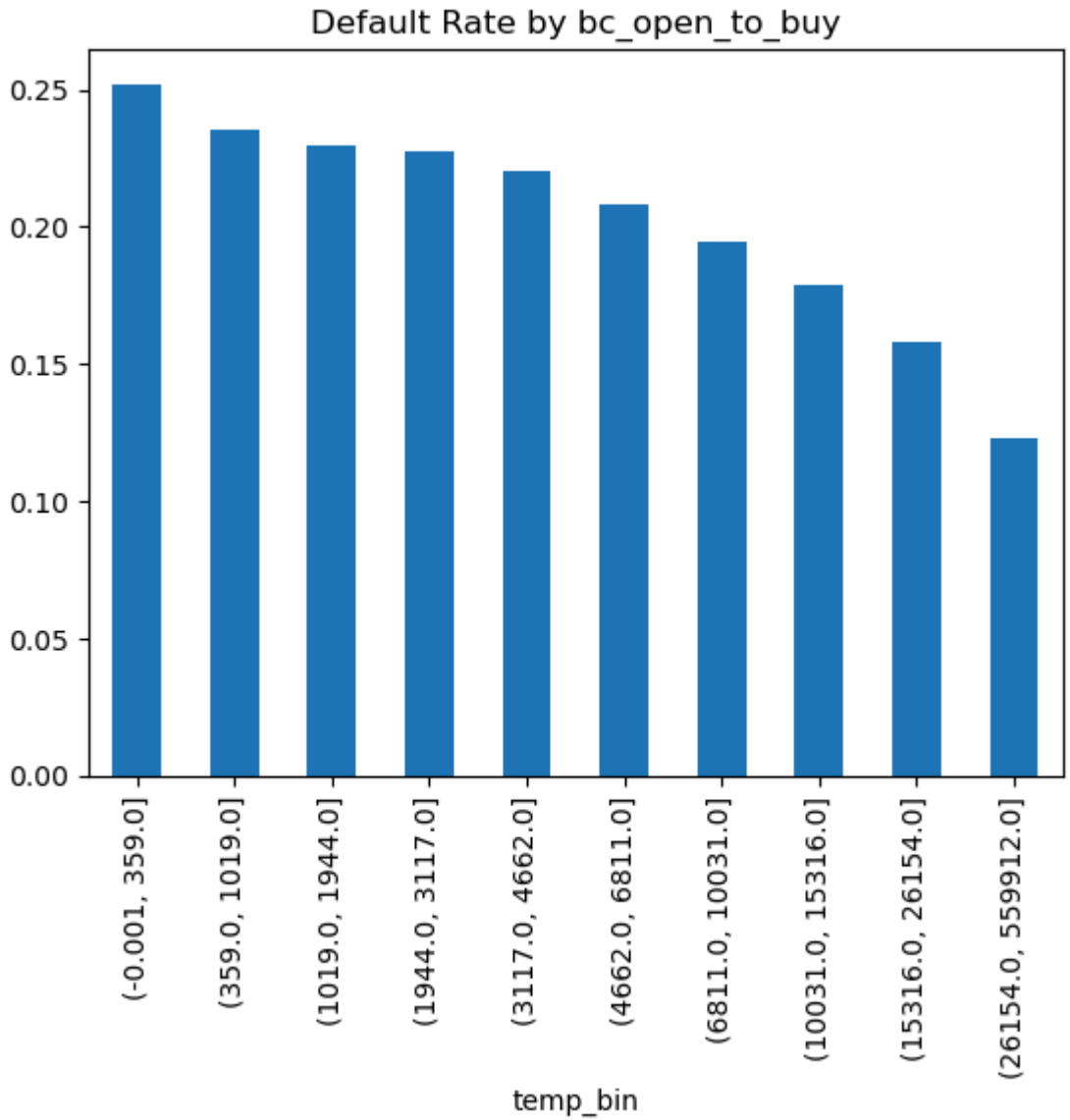

Default Rate by dti
0.30
0.25
0.20
0.15
0.10
0.05
0.00
(-1.001, 7.27]
(7.27, 10.48]
(10.48, 13.01]
(13.01, 15.32]
(15.32, 17.61]
(17.61, 19.97]
(19.97, 22.58]
(22.58, 25.66]
(25.66, 29.75]
(29.75, 999.0]
temp_bin

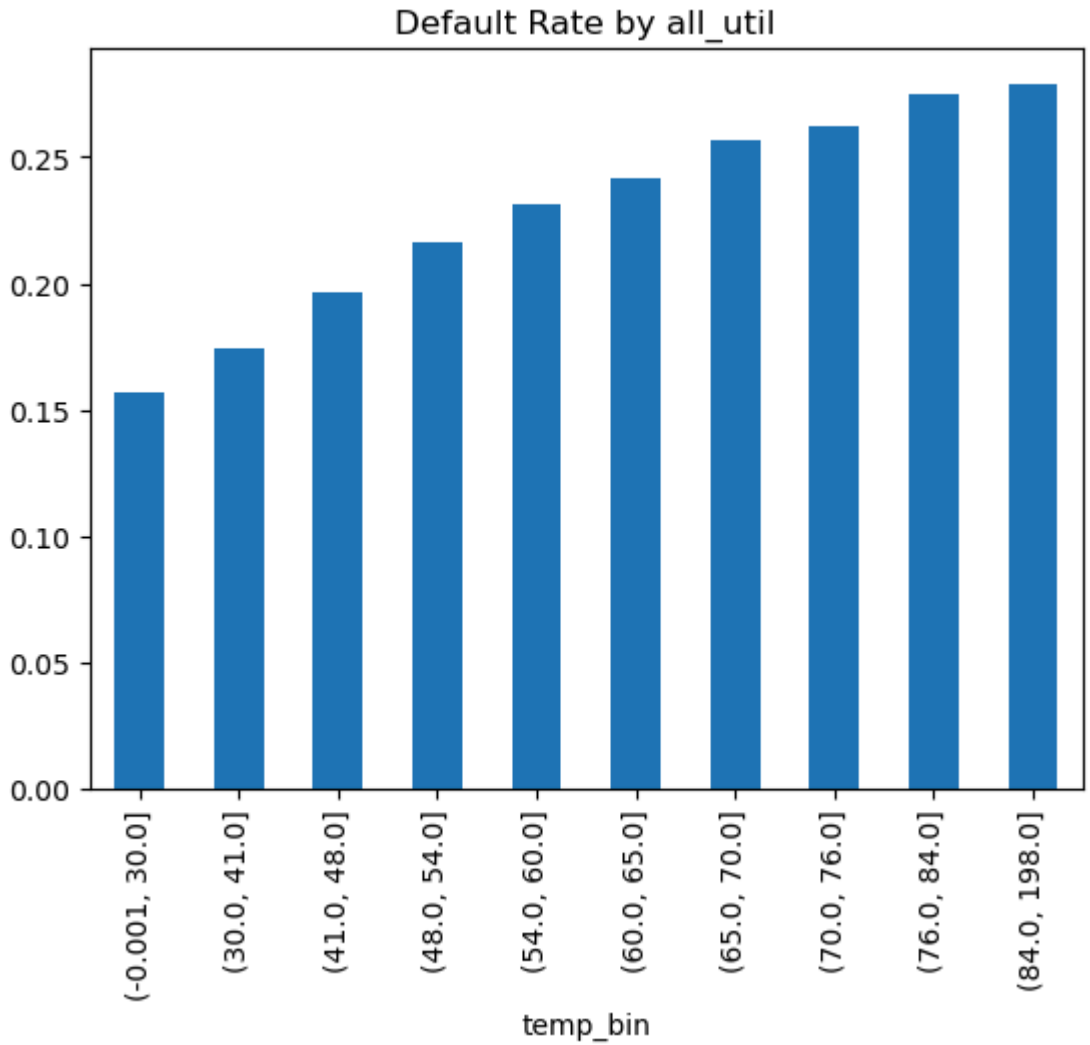

Default Rate by tot_hi_cred_lim
0.20
0.15
0.10
0.05
0.00
(-0.001, 27663.0]
(27663.0, 42200.0]
(42200.0, 57661.0]
(57661.0, 78477.0]
(78477.0, 112391.0]
(112391.0, 163919.0]
(163919.0, 220281.0]
(220281.0, 288251.0]
(288251.0, 394527.0]
(394527.0, 9999999.0]
temp_bin

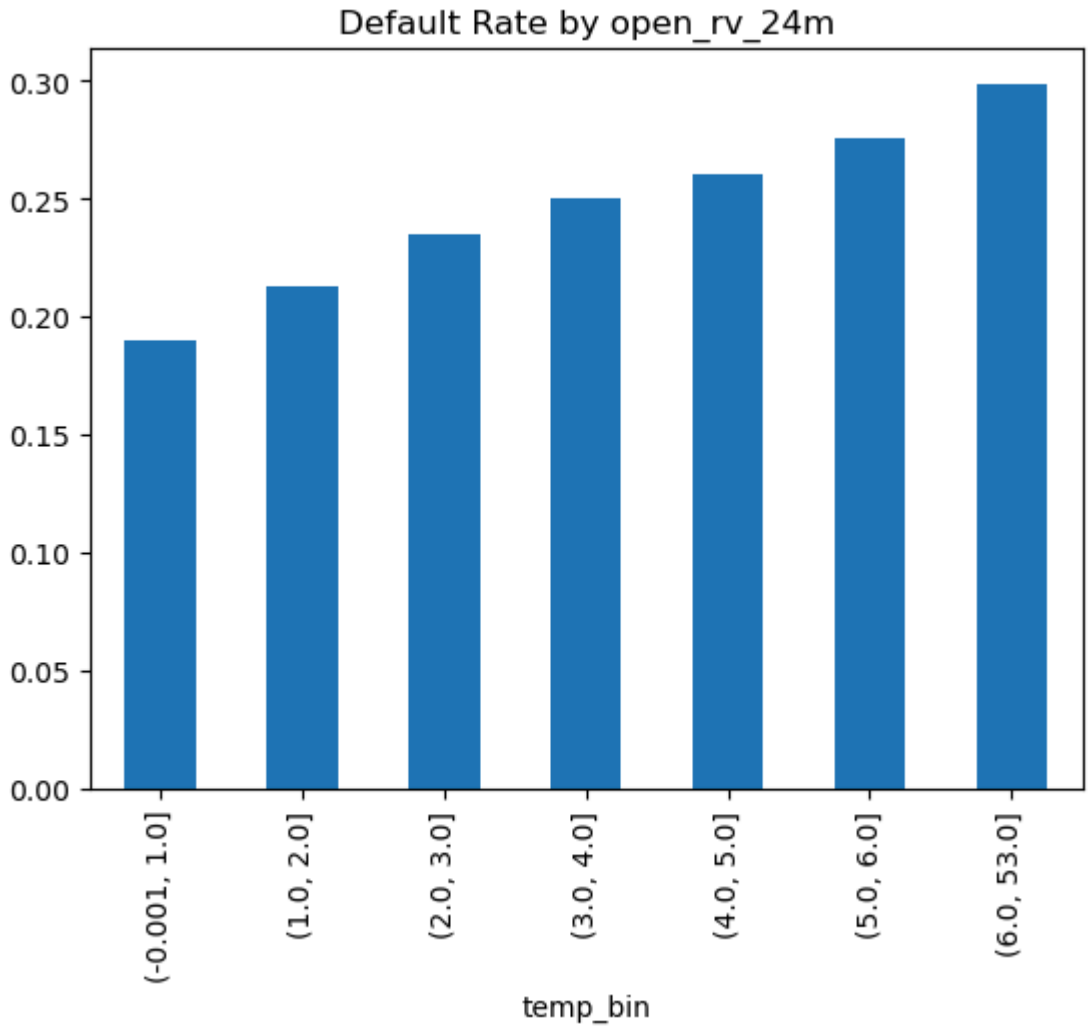

Default Rate by acc_open_past_24mths
0.30
0.25
0.20
0.15
0.10
0.05
0.00
(-0.001, 1.0]
(1.0, 2.0]
(2.0, 3.0]
(3.0, 4.0]
(4.0, 5.0]
(5.0, 6.0]
(6.0, 7.0]
(7.0, 9.0]
(9.0, 64.0]
temp_bin

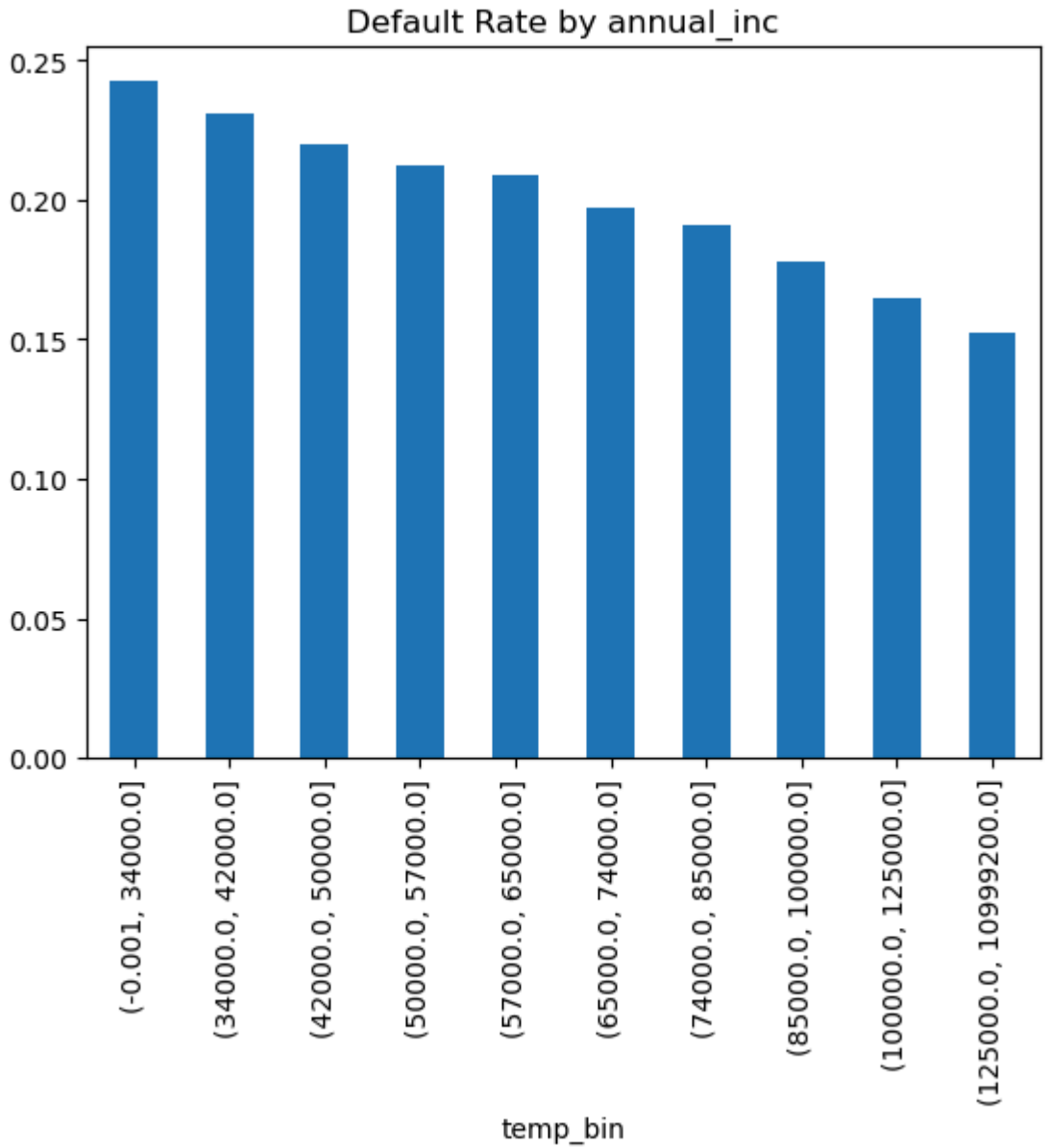

Default Rate by mort_acc
0.20
0.15
0.10
0.05
0.00
(-0.001, 1.0]
(1.0, 2.0]
(2.0, 3.0]
(3.0, 4.0]
(4.0, 51.0]
temp_bin

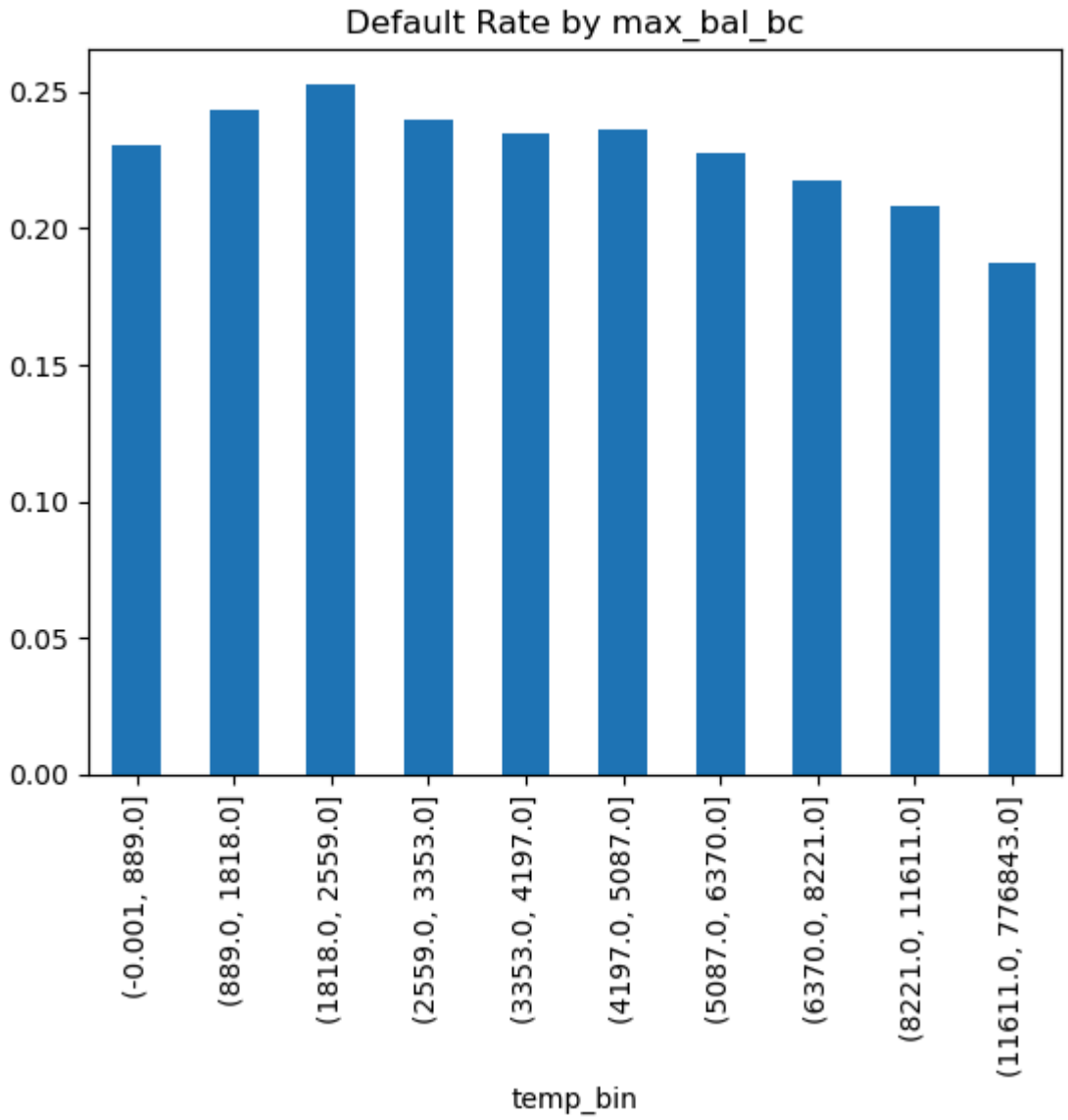

Default Rate by funded_amnt_inv
0.25
0.20
0.15
0.10
0.05
0.00
(-0.001, 4950.0]
(4950.0, 6500.0]
(6500.0, 8609.986]
(8609.986, 10000.0]
(10000.0, 12000.0]
(12000.0, 15000.0]
(15000.0, 18000.0]
(18000.0, 21000.0]
(21000.0, 28000.0]
(28000.0, 40000.0]
temp_bin

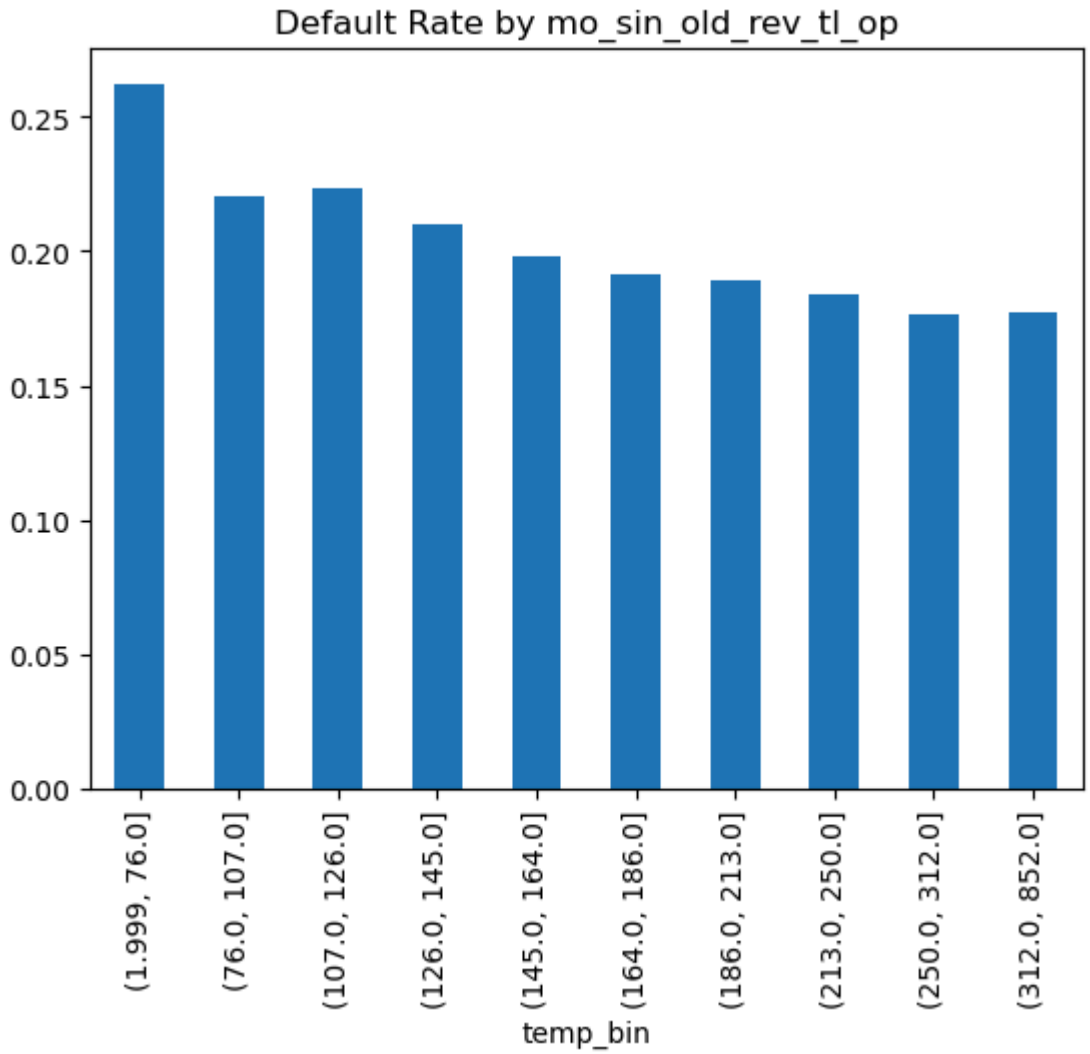

Default Rate by bc_open_to_buy
0.25
0.20
0.15
0.10
0.05
0.00
(-0.001, 359.0]
(359.0, 1019.0]
(1019.0, 1944.0]
(1944.0, 3117.0]
(3117.0, 4662.0]
(4662.0, 6811.0]
(6811.0, 10031.0]
(10031.0, 15316.0]
(15316.0, 26154.0]
(26154.0, 559912.0]
temp_bin

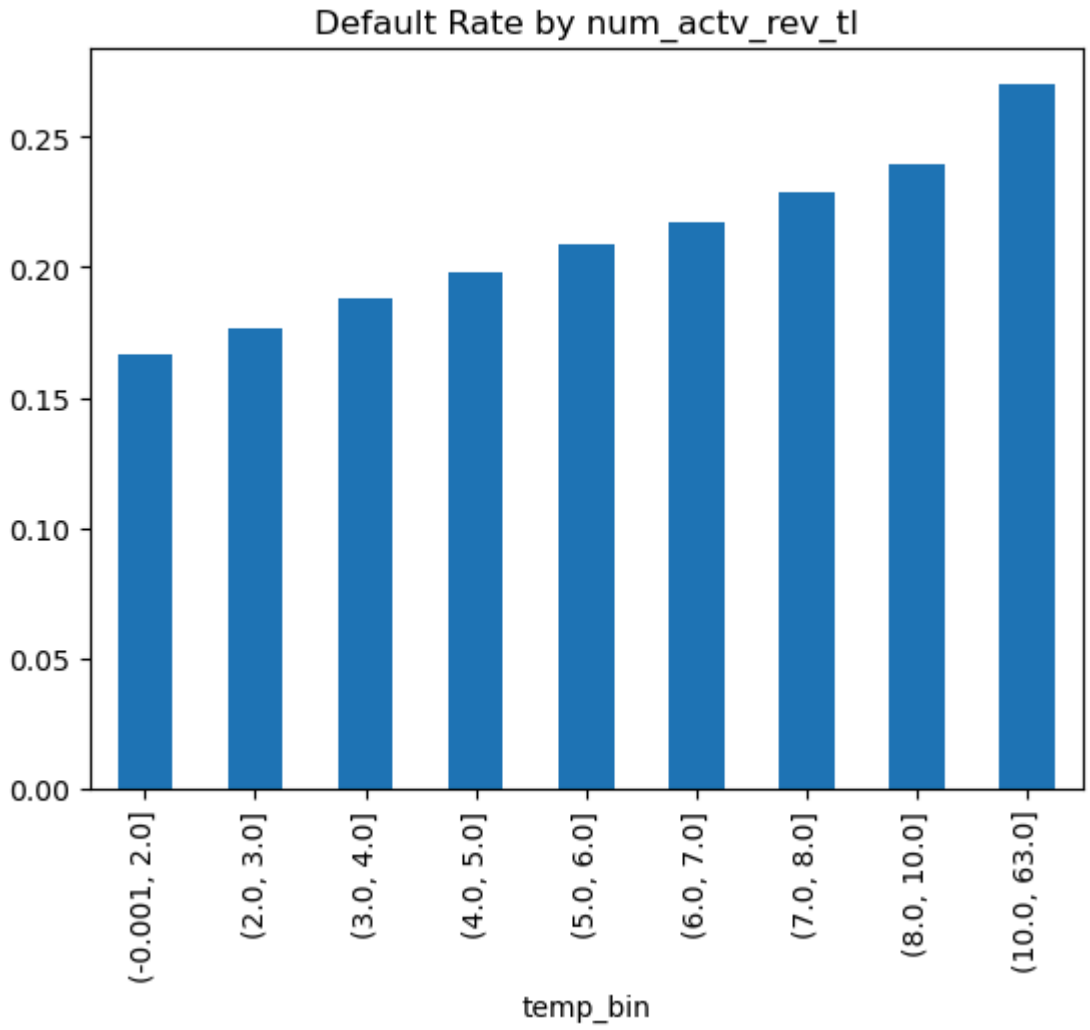

Default Rate by all_util
0.25
0.20
0.15
0.10
0.05
0.00
(-0.001, 30.0]
(30.0, 41.0]
(41.0, 48.0]
(48.0, 54.0]
(54.0, 60.0]
(60.0, 65.0]
(65.0, 70.0]
(70.0, 76.0]
(76.0, 84.0]
(84.0, 198.0]
temp_bin

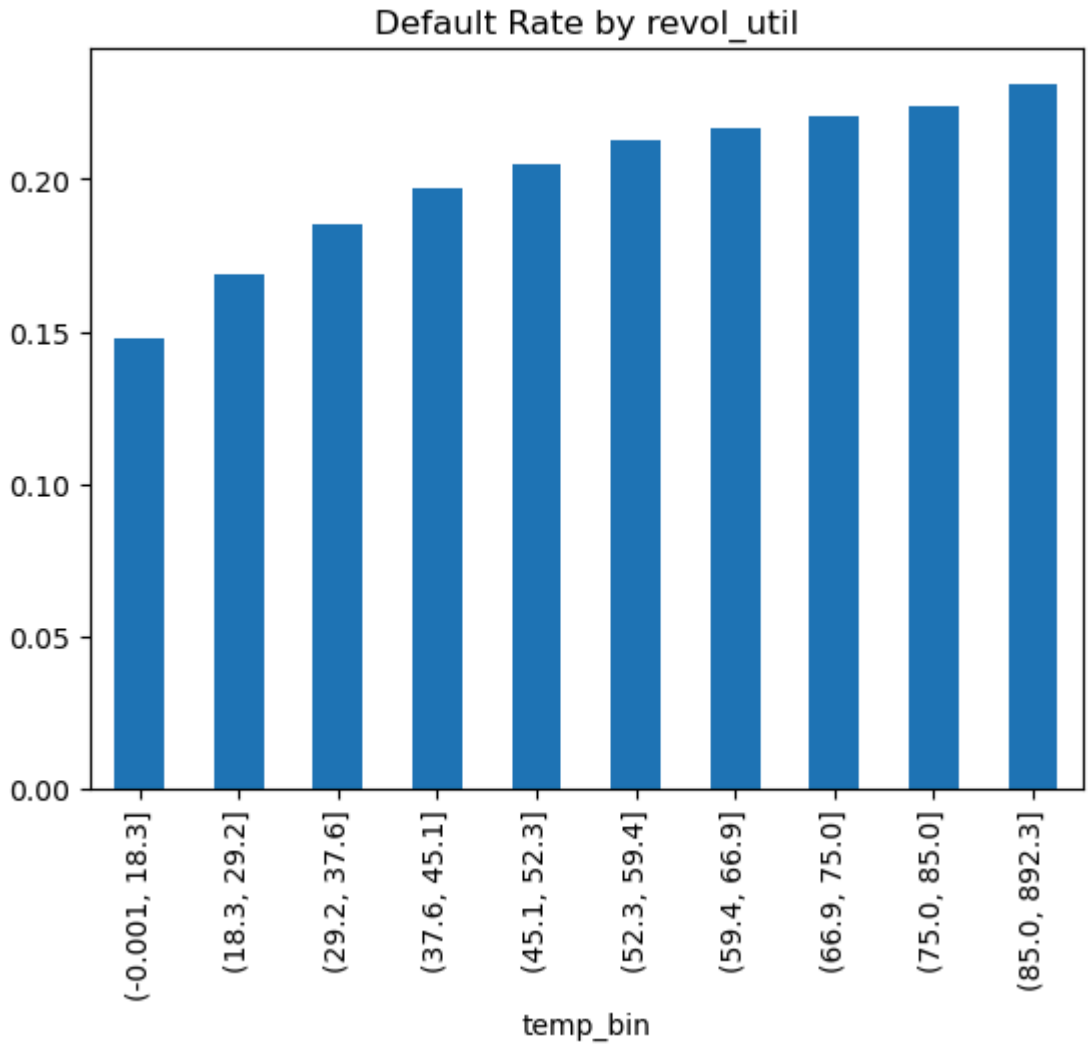

Default Rate by open_rv_24m
0.30
0.25
0.20
0.15
0.10
0.05
0.00
(-0.001, 1.0]
(1.0, 2.0]
(2.0, 3.0]
(3.0, 4.0]
(4.0, 5.0]
(5.0, 6.0]
(6.0, 53.0]
temp_bin


Default Rate by annual_inc
0.25
0.20
0.15
0.10
0.05
0.00
(-0.001, 34000.0]
(34000.0, 42000.0]
(42000.0, 50000.0]
(50000.0, 57000.0]
(57000.0, 65000.0]
(65000.0, 74000.0]
(74000.0, 85000.0]
(85000.0, 100000.0]
(100000.0, 125000.0]
(125000.0, 10999200.0]
temp_bin


Default Rate by max_bal_bc
0.25
0.20
0.15
0.10
0.05
0.00
(-0.001, 889.0]
(889.0, 1818.0]
(1818.0, 2559.0]
(2559.0, 3353.0]
(3353.0, 4197.0]
(4197.0, 5087.0]
(5087.0, 6370.0]
(6370.0, 8221.0]
(8221.0, 11611.0]
(11611.0, 776843.0]
temp_bin


Default Rate by mo_sin_old_rev_tl_op
0.25
0.20
0.15
0.10
0.05
0.00
(1.999, 76.0]
(76.0, 107.0]
(107.0, 126.0]
(126.0, 145.0]
(145.0, 164.0]
(164.0, 186.0]
(186.0, 213.0]
(213.0, 250.0]
(250.0, 312.0]
(312.0, 852.0]
temp_bin

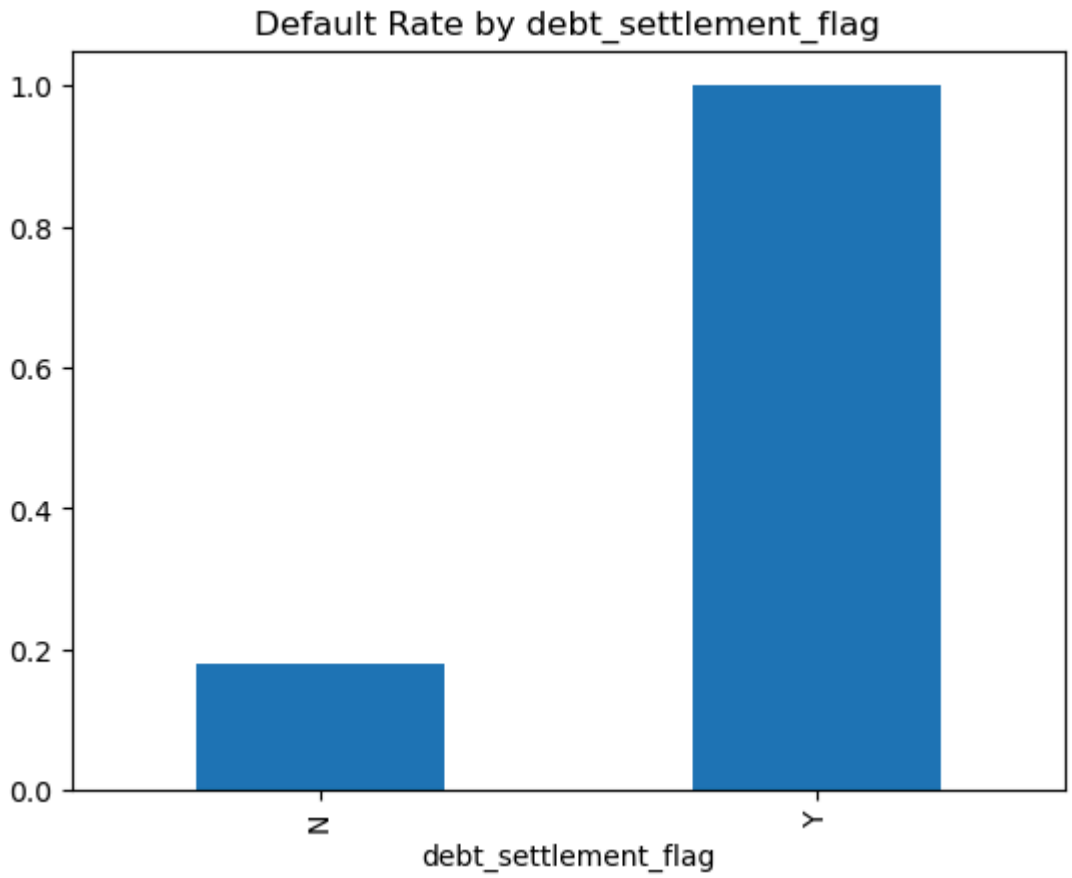

Default Rate by num_actv_rev_tl
0.25
0.20
0.15
0.10
0.05
0.00
(-0.001, 2.0]
(2.0, 3.0]
(3.0, 4.0]
(4.0, 5.0]
(5.0, 6.0]
(6.0, 7.0]
(7.0, 8.0]
(8.0, 10.0]
(10.0, 63.0]
temp_bin

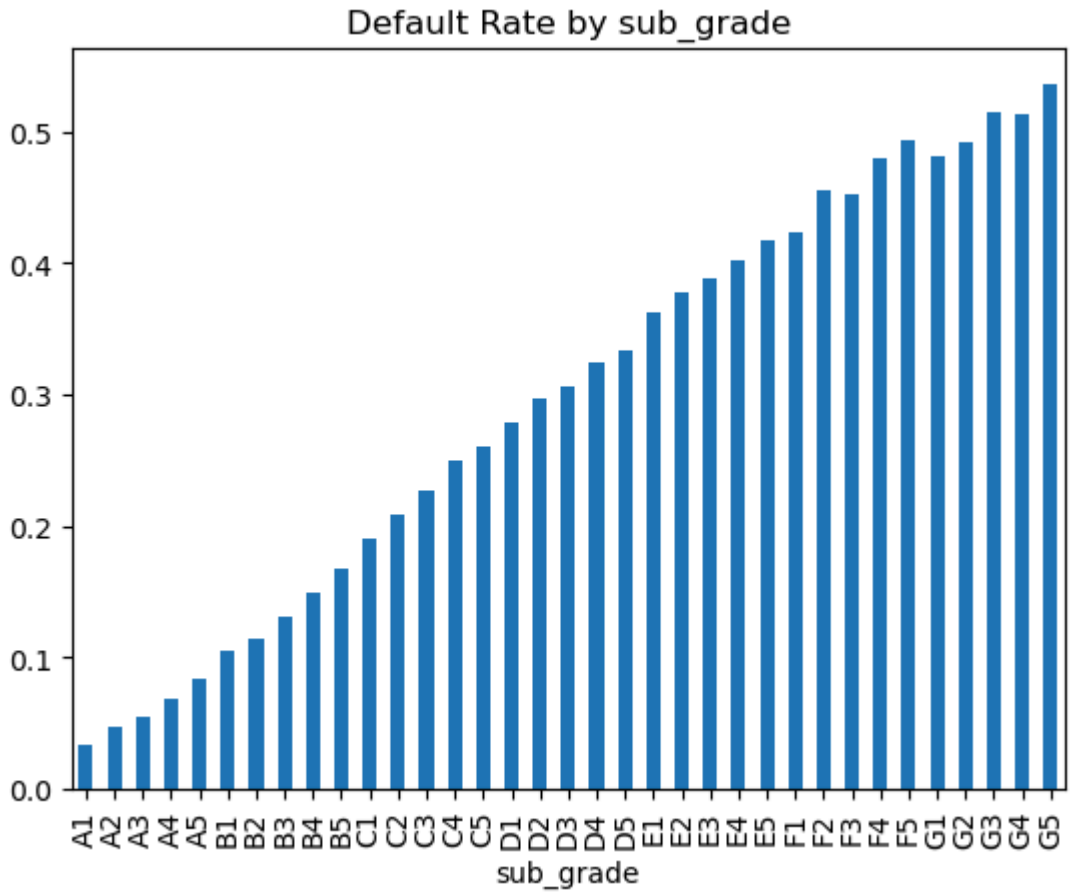


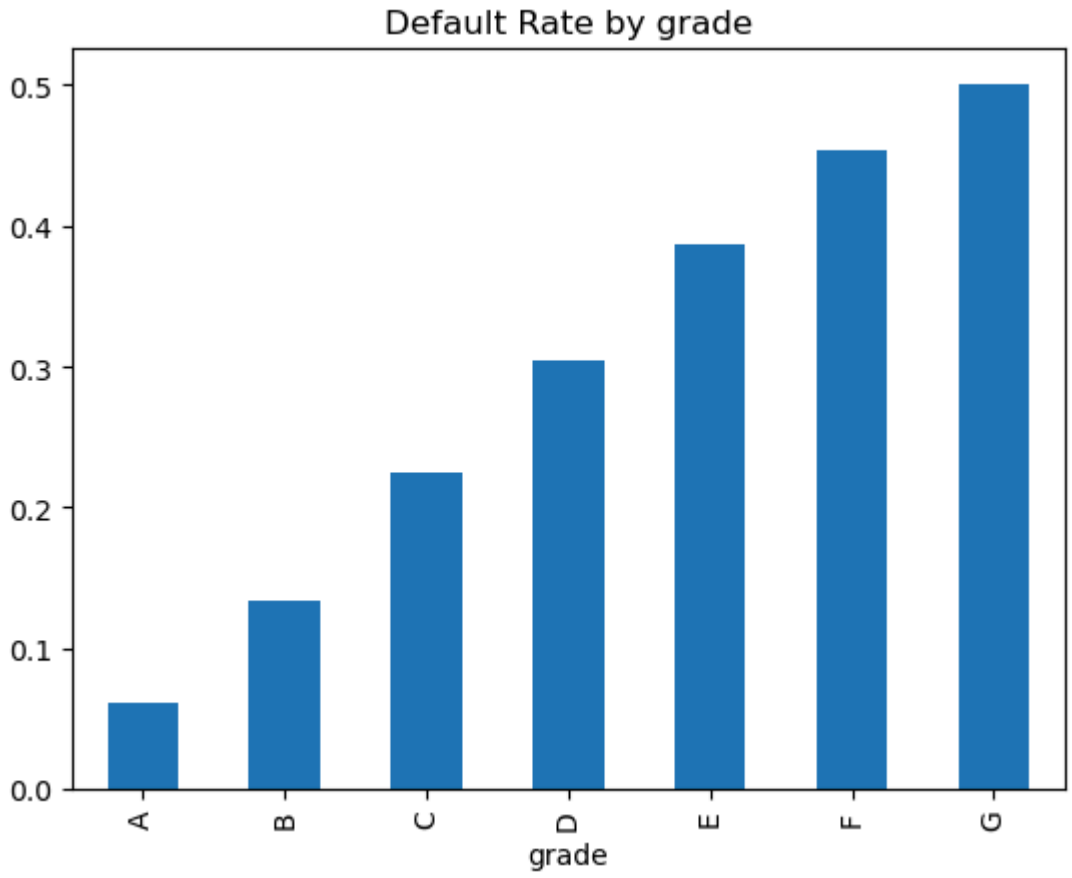
Default Rate by Categorical Variable

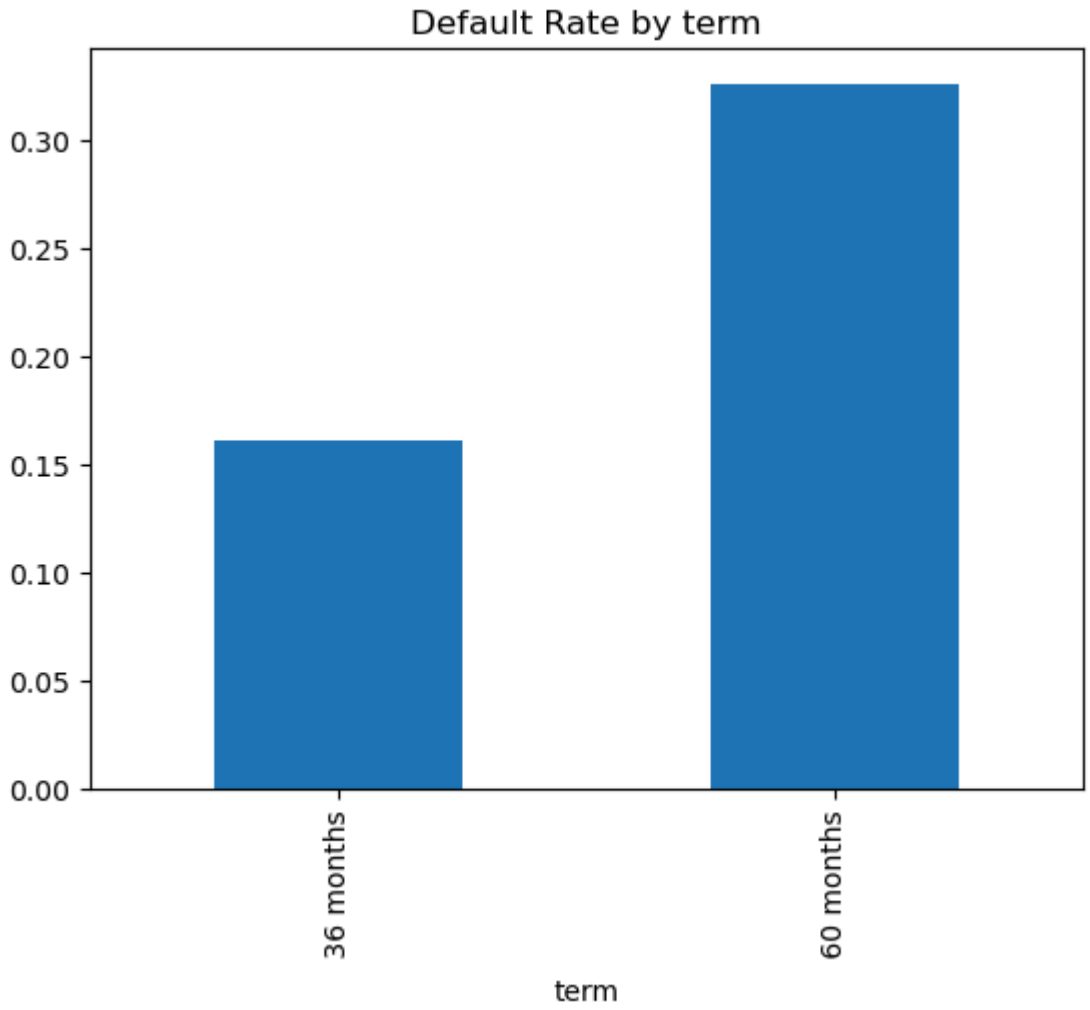
In [14]:
```
for col in cat_cols:

    default_rate = df.groupby(col)['target'].mean()

    default_rate.plot(kind='bar')
```

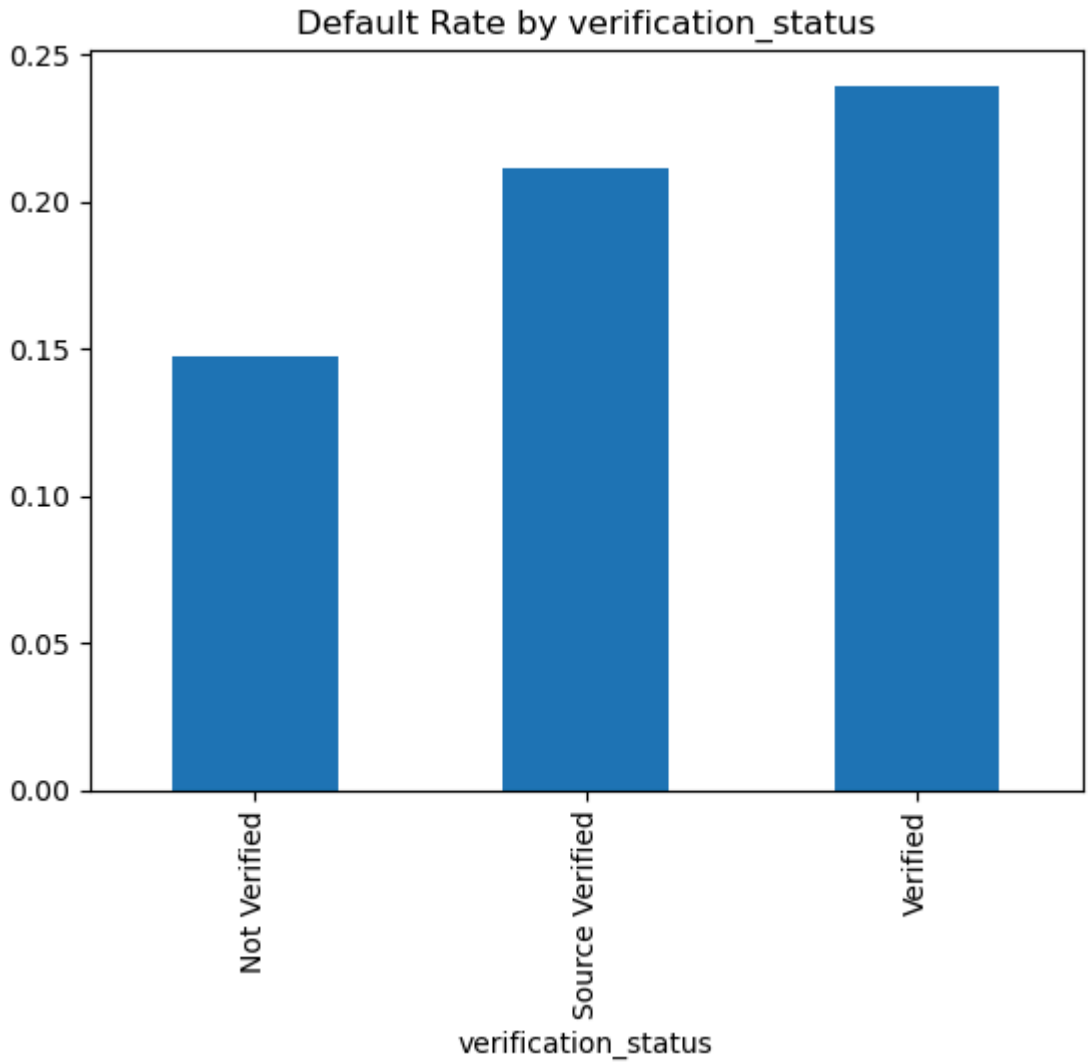
```
plt.title(f"Default Rate by {col}")

plt.show()
```

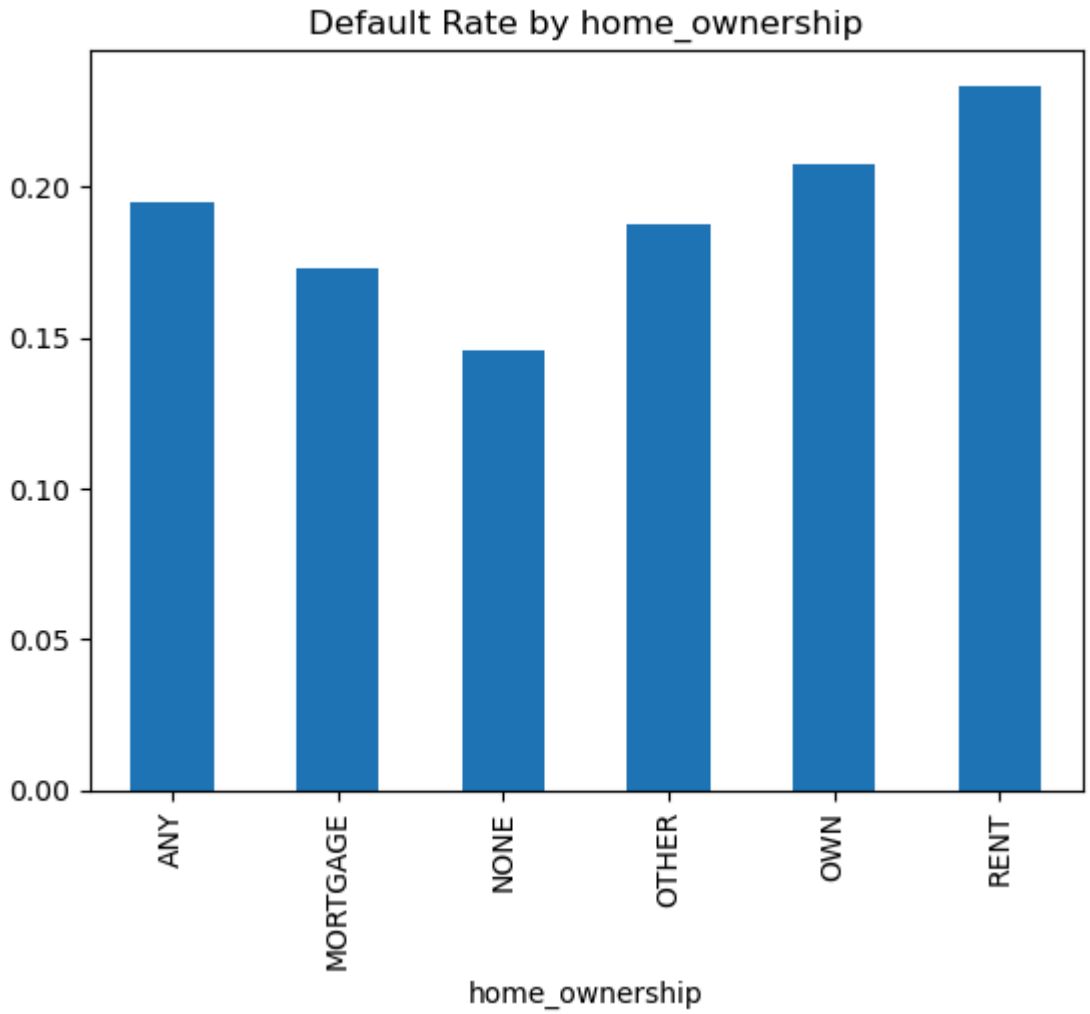


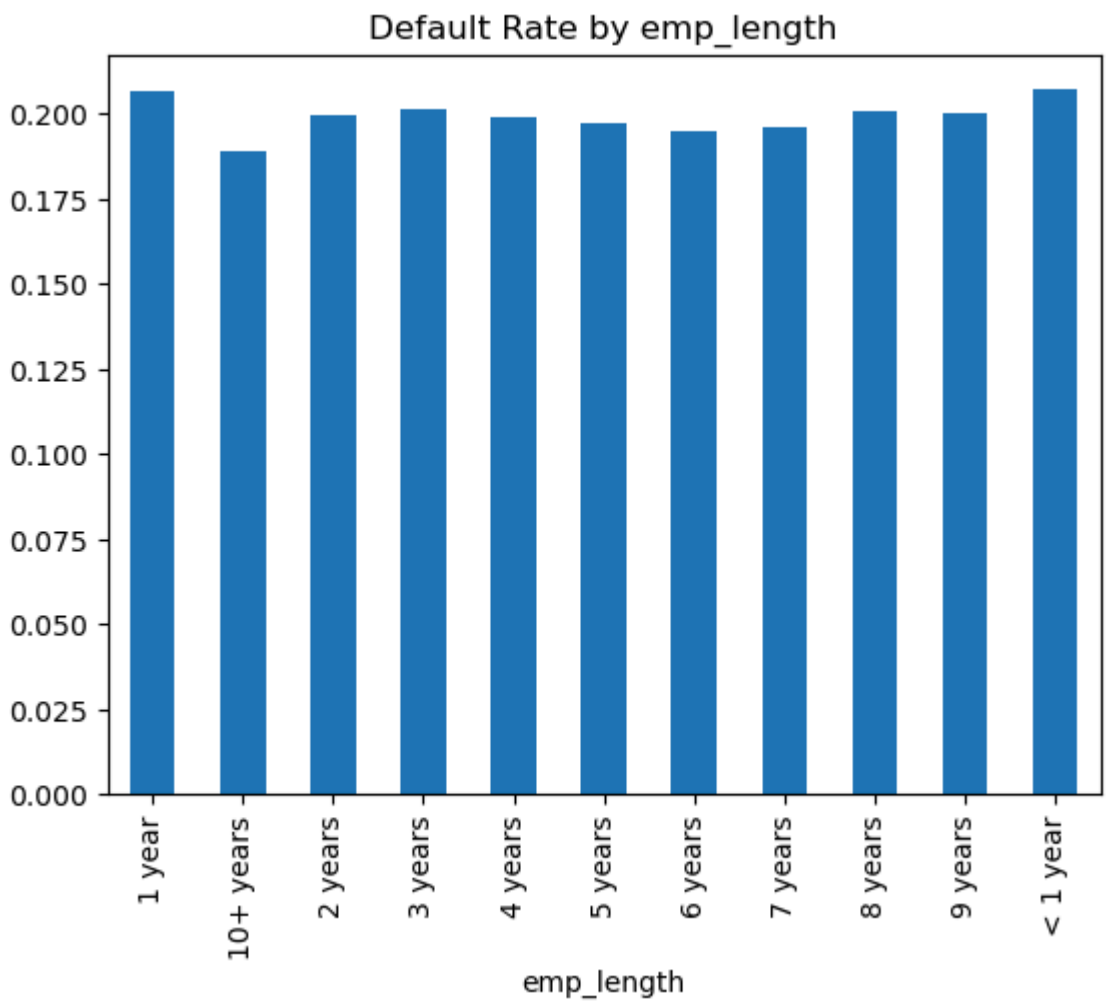

Default Rate by sub_grade
0.0
0.1
0.2
0.3
0.4
0.5
A1 A2 A3 A4 A5 B1 B2 B3 B4 B5 C1 C2 C3 C4 C5 D1 D2 D3 D4 D5 E1 E2 E3 E4 E5 F1 F2 F3 F4 F5 G1 G2 G3 G4 G5
sub_grade


Default Rate by grade
0.5
0.4
0.3
0.2
0.1
0.0
A
B
C
D
E
F
G
grade


Default Rate by term
0.30
0.25
0.20
0.15
0.10
0.05
0.00
36 months
60 months
term


Default Rate by verification_status
0.25
0.20
0.15
0.10
0.05
0.00
Not Verified
Source Verified
Verified
verification_status


Default Rate by home_ownership
0.20
0.15
0.10
0.05
0.00
ANY
MORTGAGE
NONE
OTHER
OWN
RENT
home_ownership



WOE Binning

In [15]:
```
def woe_iv(data,feature,target):

    table = pd.crosstab(data[feature],data[target])

    table.columns=["Good","Bad"]
```

```
    table["Good"]+=0.5
    table["Bad"]+=0.5

    table["Dist_Good"]=table["Good"]/table["Good"].sum()
    table["Dist_Bad"]=table["Bad"]/table["Bad"].sum()

    table["WOE"]=np.log(table["Dist_Good"]/table["Dist_Bad"])

    table["IV"]=(table["Dist_Good"]-table["Dist_Bad"])*table["WOE"]

    iv = table["IV"].sum()

    return table,iv
```

Numerical Binning

In [16]:
```
for col in num_cols:

    train_df[col+"_bin"] = pd.qcut(

        train_df[col],
        10,
        duplicates='drop'
    )

    val_df[col+"_bin"] = pd.cut(

        val_df[col],
        bins=train_df[col+"_bin"].cat.categories
    )
```

Monotonic Bining of the Numerical Variable

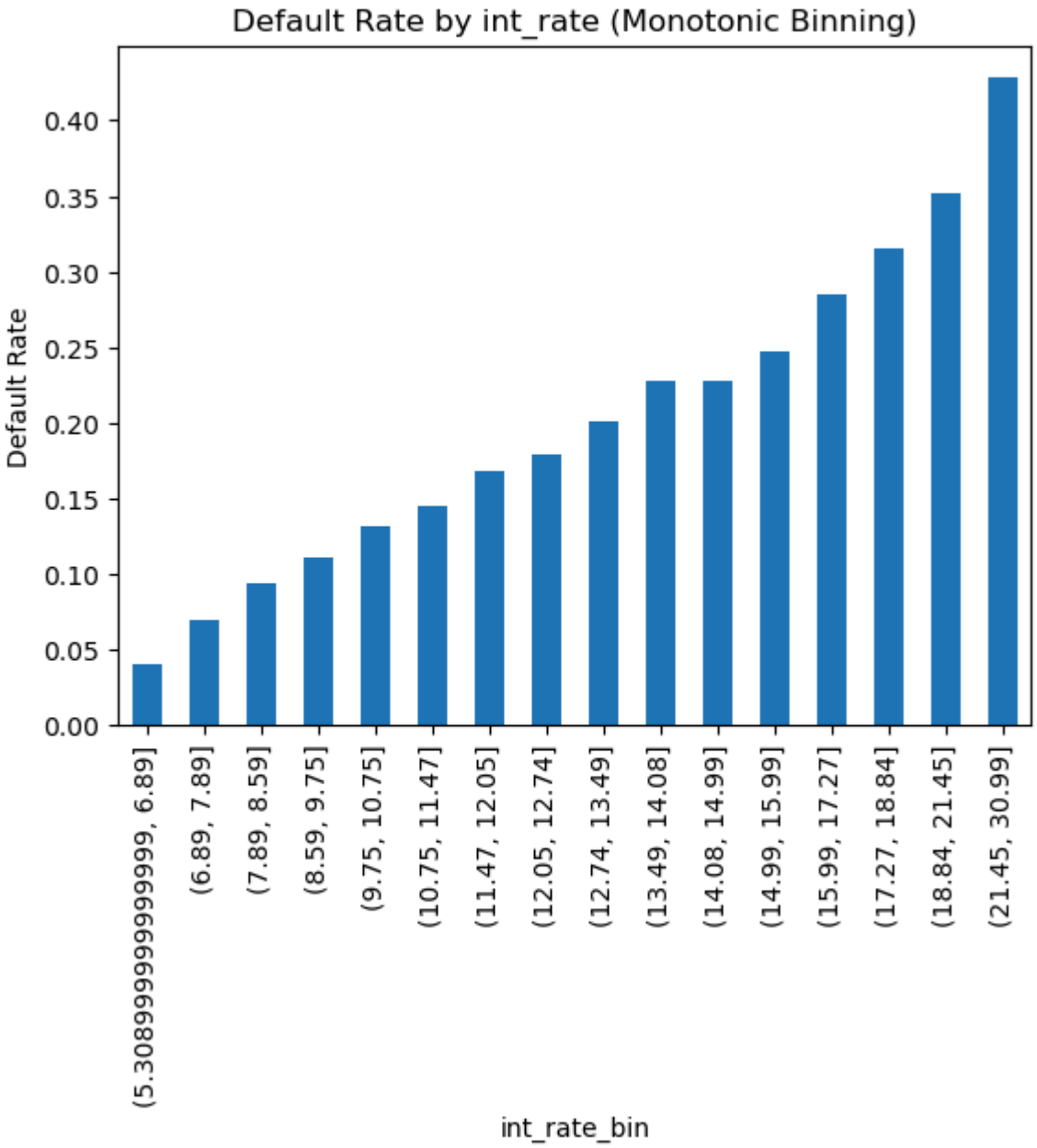
In [17]:
```
from scipy.stats import spearmanr


def monotonic_binning(df, variable, target, max_bins=20):
```

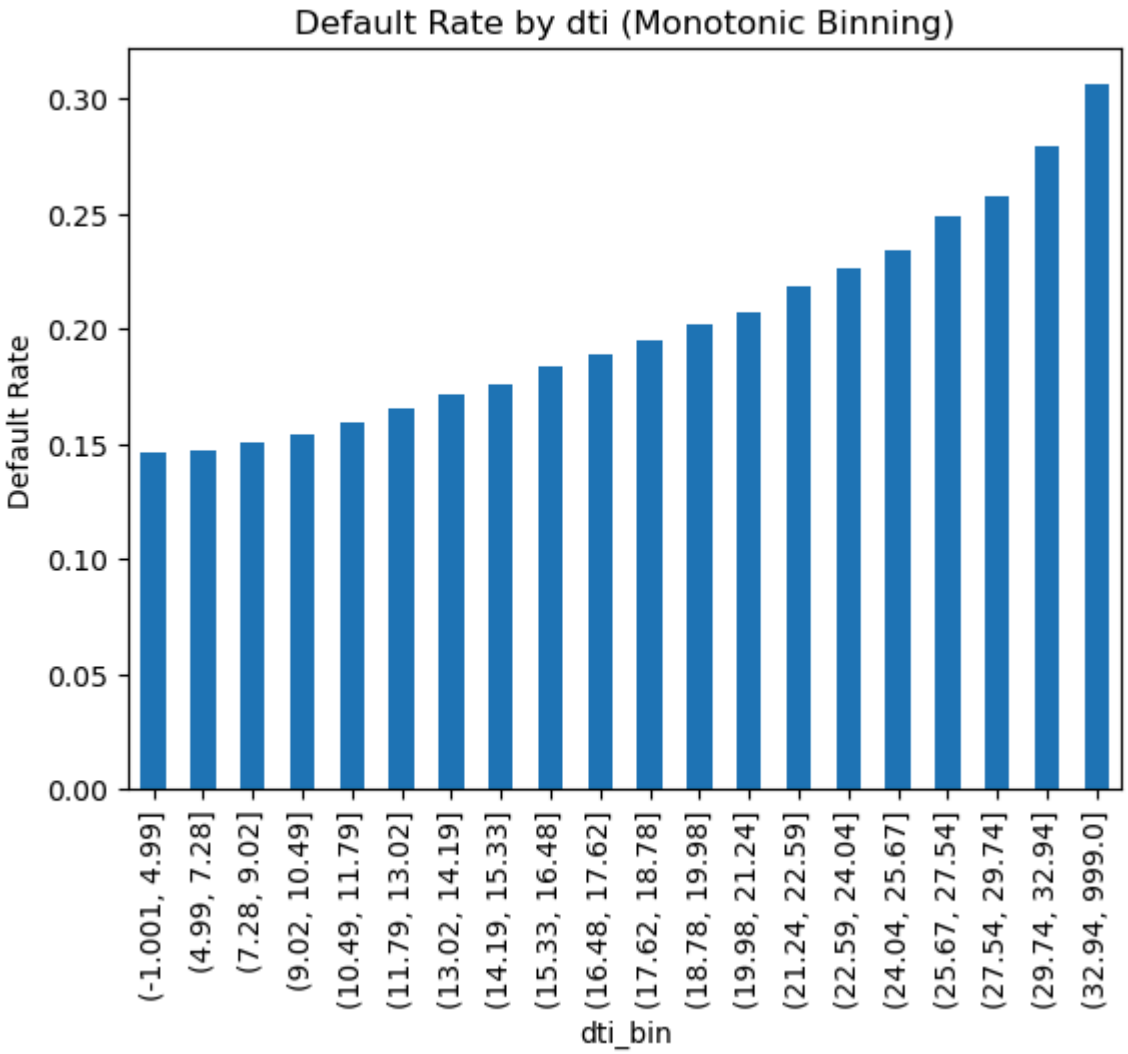
```
    df2 = df[[variable, target]].copy()
    df2 = df2.dropna()

    r = 0
    bins = max_bins

    while abs(r) < 1 and bins > 2:

        try:

            df2['bin'] = pd.qcut(df2[variable], bins, duplicates='drop')

            grouped = df2.groupby('bin')

            mean_x = grouped[variable].mean()
            mean_y = grouped[target].mean()

            r, _ = spearmanr(mean_x, mean_y)

            bins -= 1

        except:
            bins -= 1

    return df2['bin'], bins


# Apply Monotonic Binning

binning_dict = {}

for col in num_cols:

    train_df[col+"_bin"], bins = monotonic_binning(
        train_df,
        col,
        'target'
    )

    # store bin edges for validation
    bin_edges = train_df[col+"_bin"].cat.categories
```

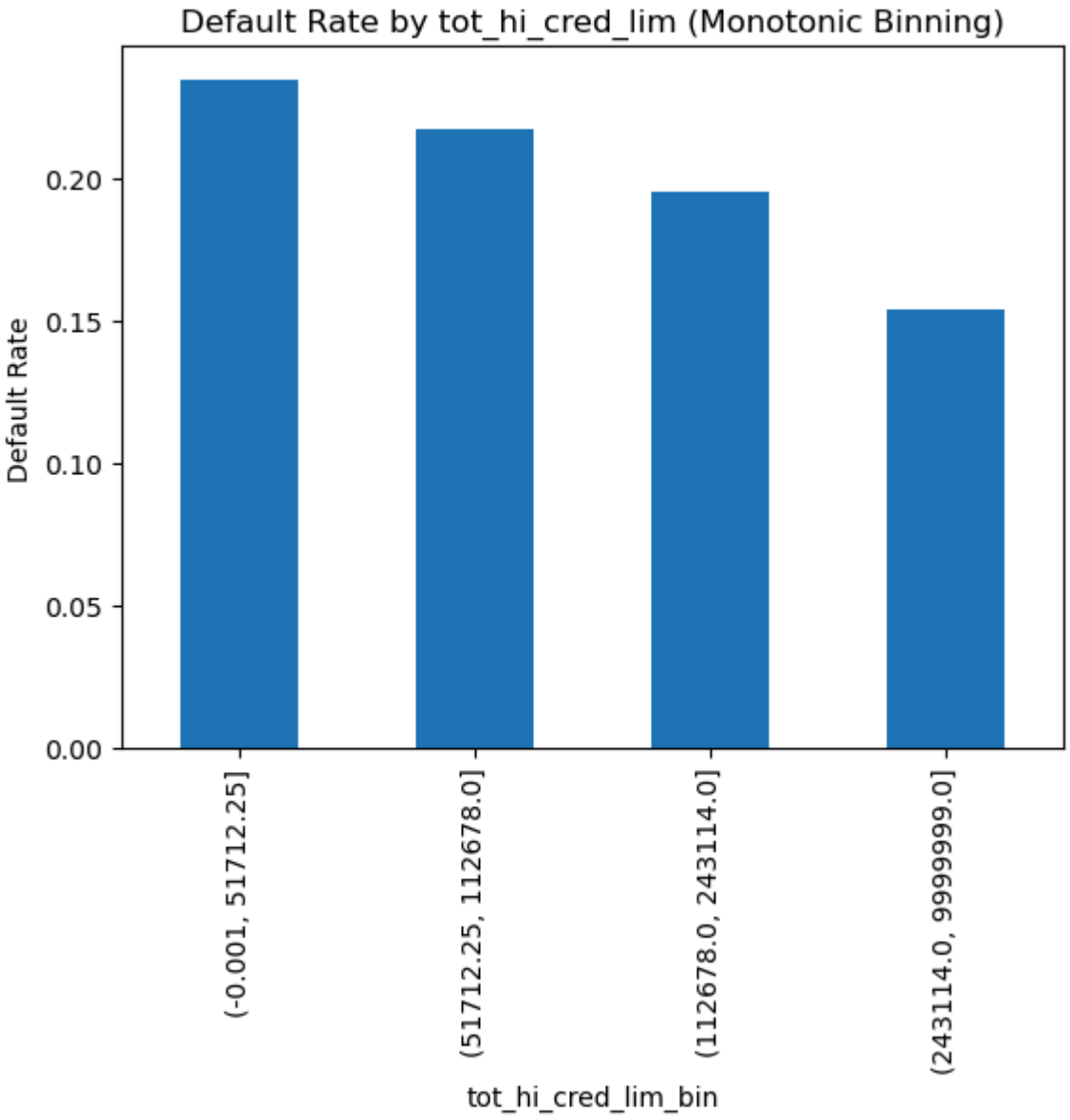
```
    binning_dict[col] = bin_edges

    val_df[col+"_bin"] = pd.cut(
        val_df[col],
        bins=bin_edges
    )
```

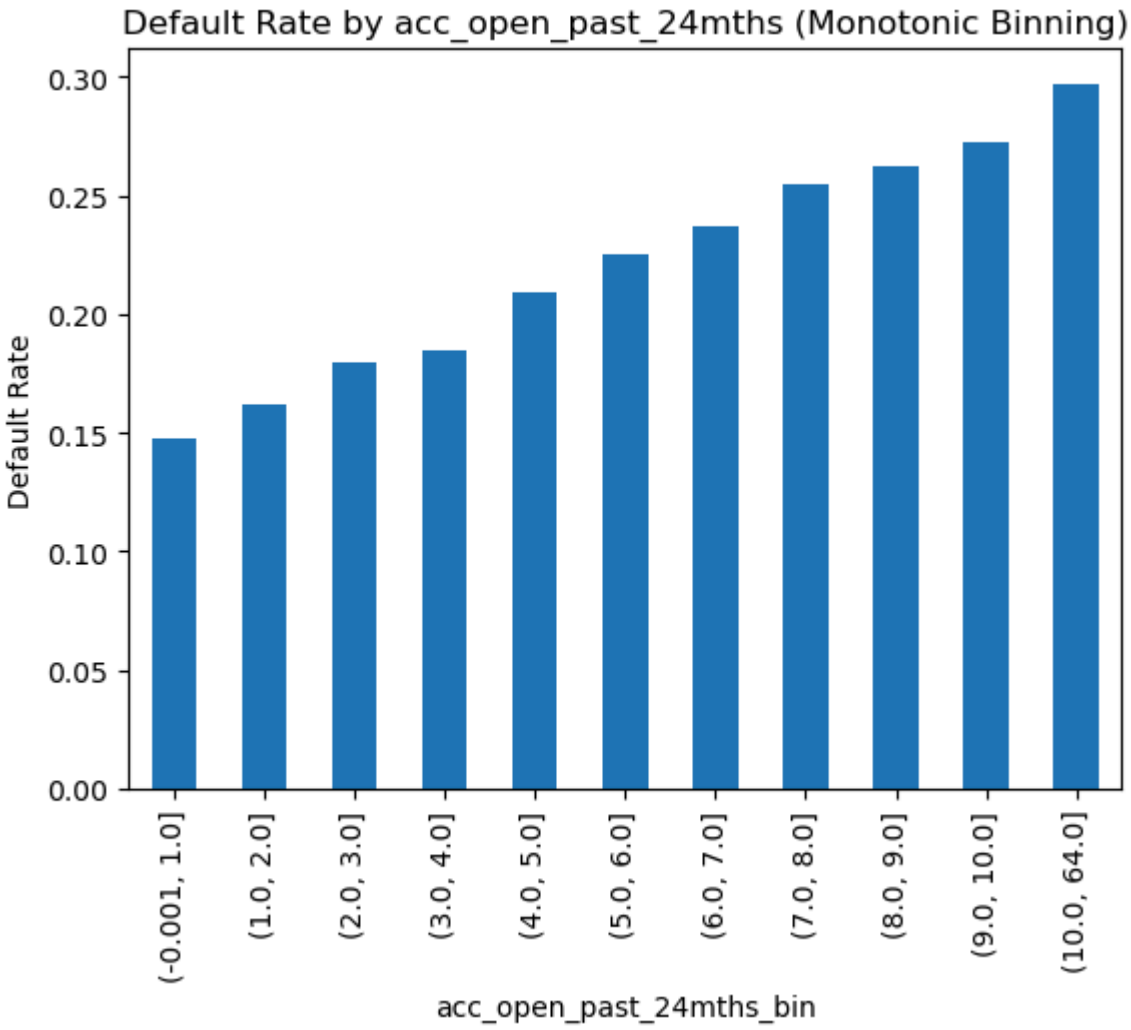
Visualize Monotonic Default Rate

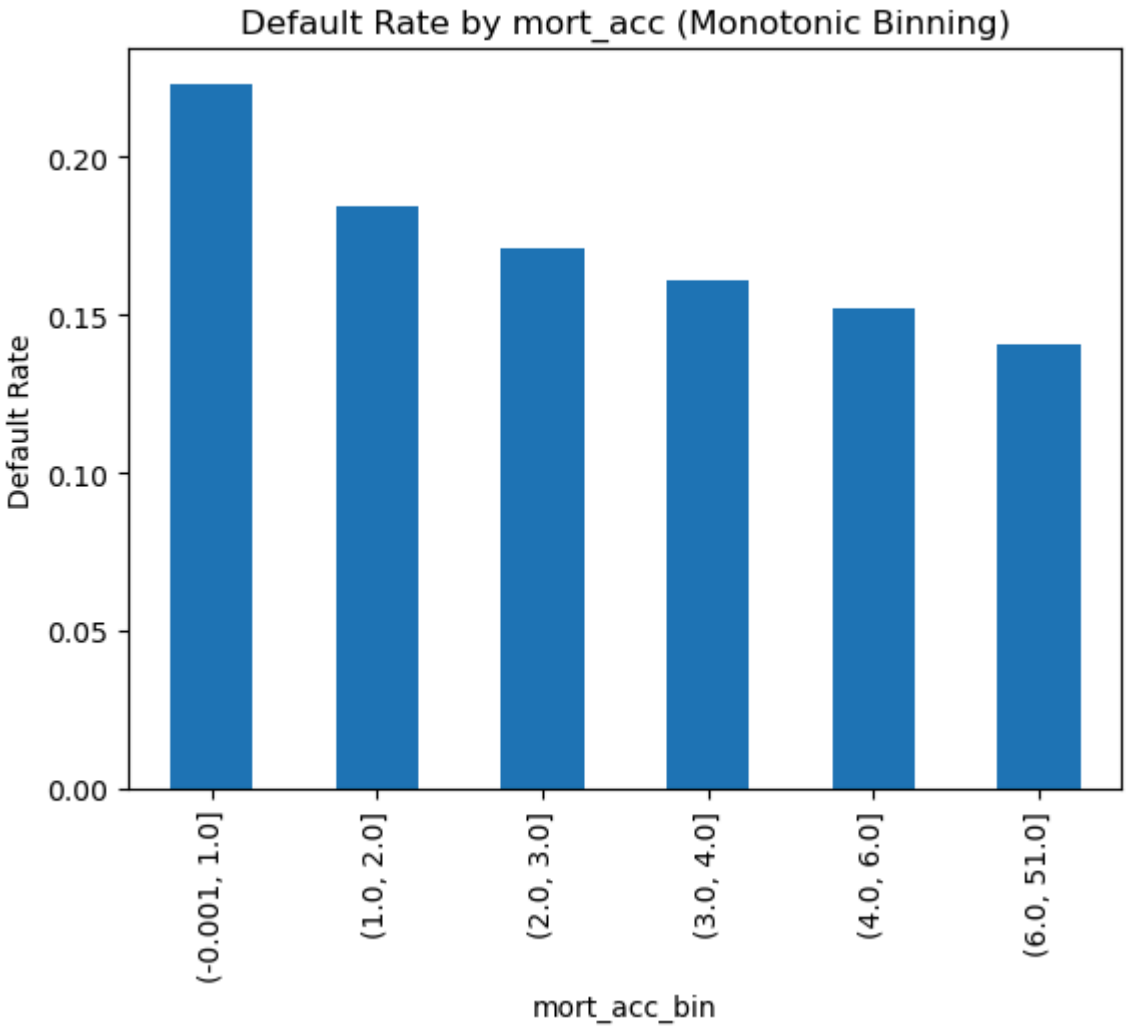
In [18]:
```
for col in num_cols:

    temp = train_df.groupby(col+"_bin")['target'].mean()

    temp.plot(kind='bar')

    plt.title(f"Default Rate by {col} (Monotonic Binning)")

    plt.ylabel("Default Rate")

    plt.show()
```

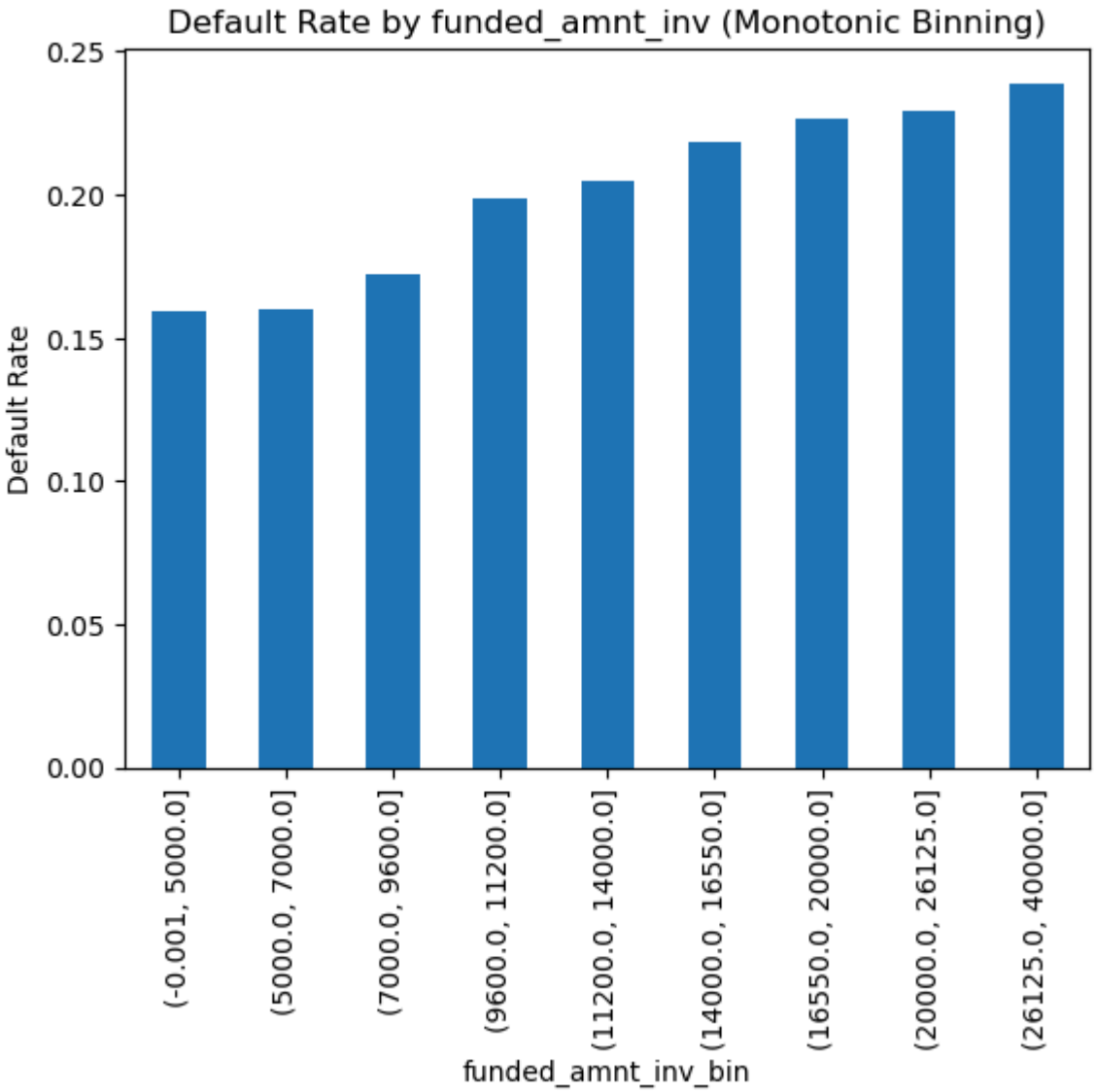

Default Rate by int_rate (Monotonic Binning)
Default Rate
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
(5.3089999999999999, 6.89]
(6.89, 7.89]
(7.89, 8.59]
(8.59, 9.75]
(9.75, 10.75]
(10.75, 11.47]
(11.47, 12.05]
(12.05, 12.74]
(12.74, 13.49]
(13.49, 14.08]
(14.08, 14.99]
(14.99, 15.99]
(15.99, 17.27]
(17.27, 18.84]
(18.84, 21.45]
(21.45, 30.99]
int_rate_bin

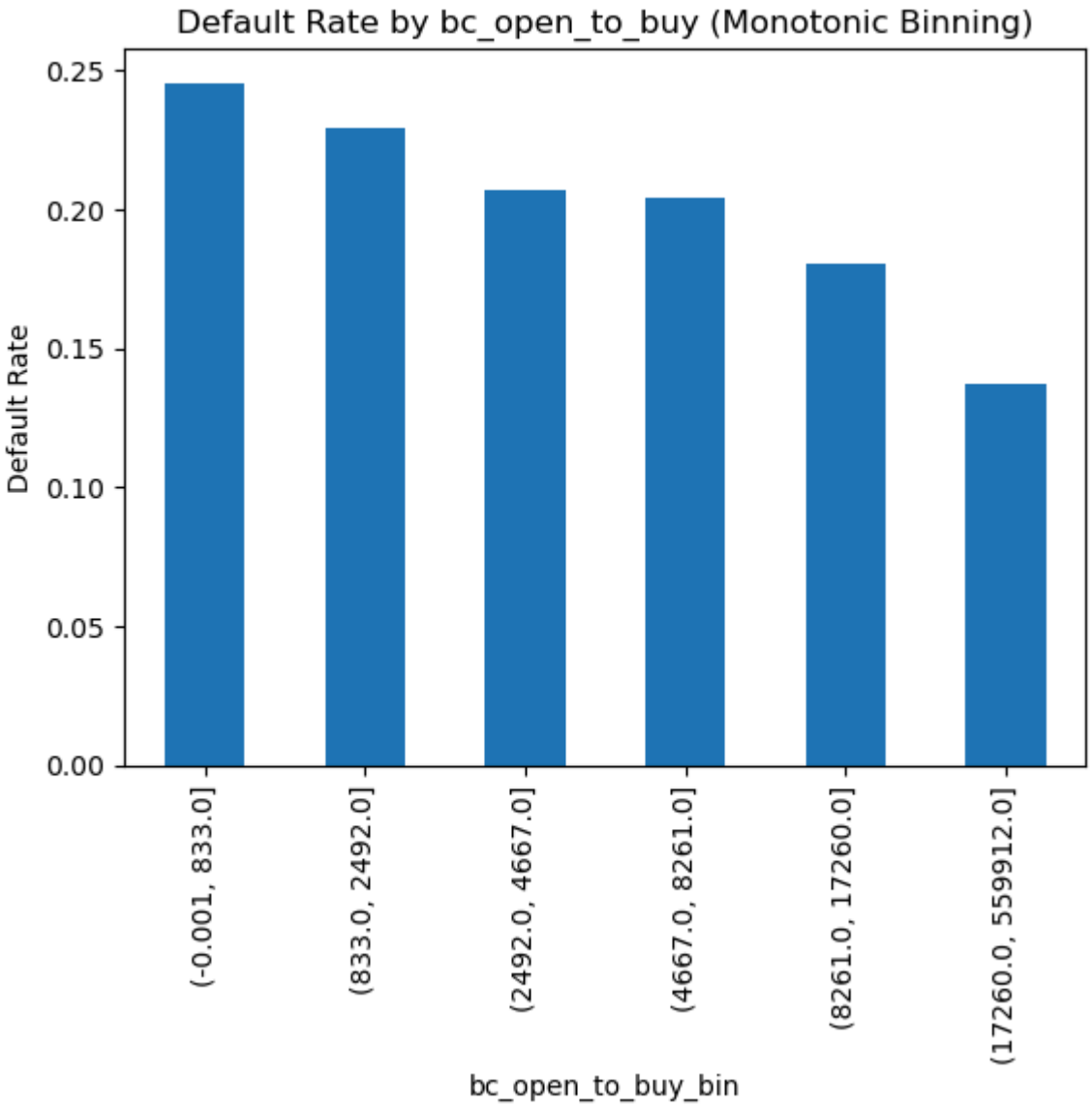

Default Rate by dti (Monotonic Binning)
Default Rate
0.00
0.05
0.10
0.15
0.20
0.25
0.30
(-1.001, 4.99]
(4.99, 7.28]
(7.28, 9.02]
(9.02, 10.49]
(10.49, 11.79]
(11.79, 13.02]
(13.02, 14.19]
(14.19, 15.33]
(15.33, 16.48]
(16.48, 17.62]
(17.62, 18.78]
(18.78, 19.98]
(19.98, 21.24]
(21.24, 22.59]
(22.59, 24.04]
(24.04, 25.67]
(25.67, 27.54]
(27.54, 29.74]
(29.74, 32.94]
(32.94, 999.0]
dti_bin

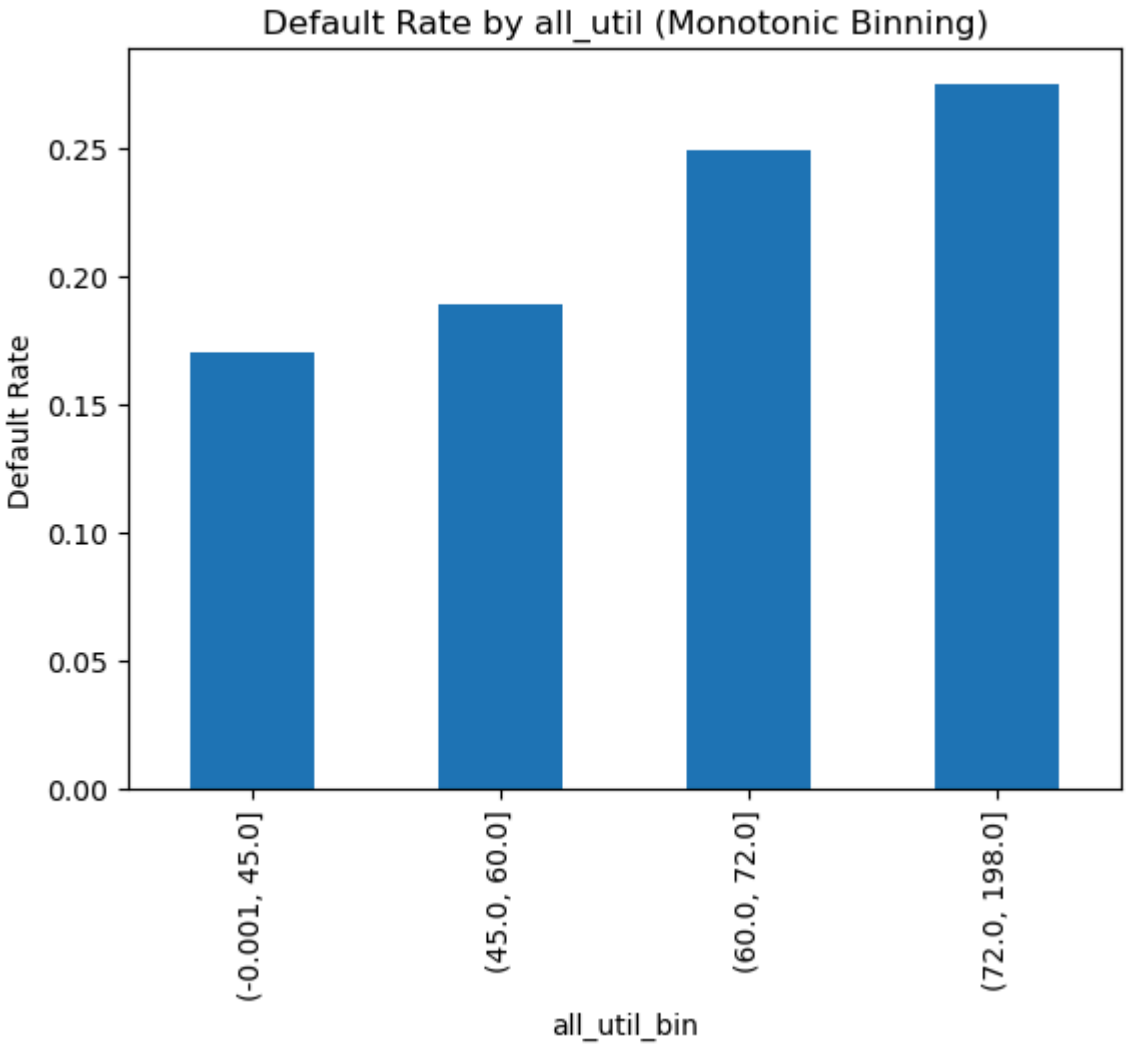

Default Rate by tot_hi_cred_lim (Monotonic Binning)
Default Rate
0.00
0.05
0.10
0.15
0.20
(-0.001, 51712.25]
(51712.25, 112678.0]
(112678.0, 243114.0]
(243114.0, 9999999.0]
tot_hi_cred_lim_bin

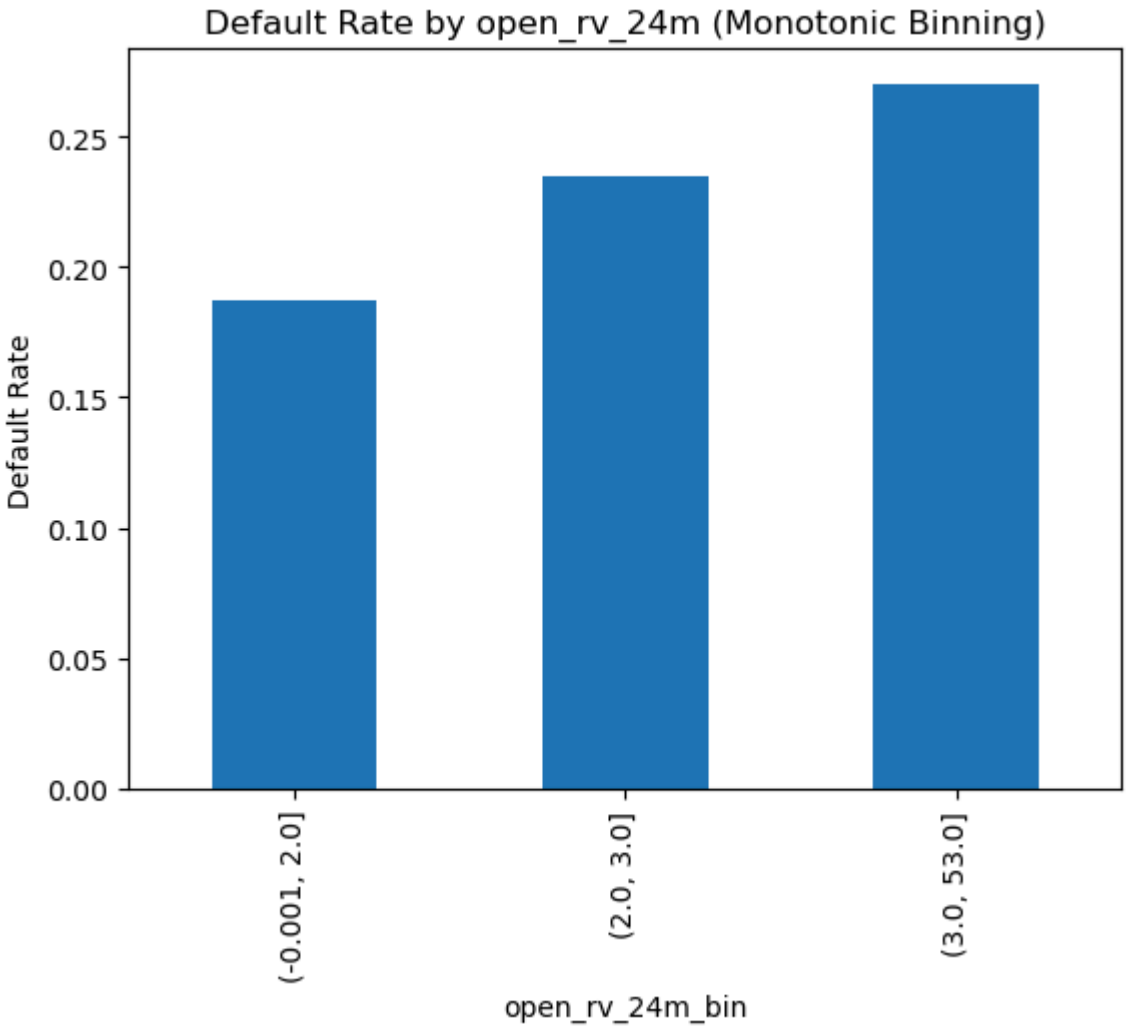

Default Rate by acc_open_past_24mths (Monotonic Binning)
Default Rate
0.30
0.25
0.20
0.15
0.10
0.05
0.00
(-0.001, 1.0]
(1.0, 2.0]
(2.0, 3.0]
(3.0, 4.0]
(4.0, 5.0]
(5.0, 6.0]
(6.0, 7.0]
(7.0, 8.0]
(8.0, 9.0]
(9.0, 10.0]
(10.0, 64.0]
acc_open_past_24mths_bin

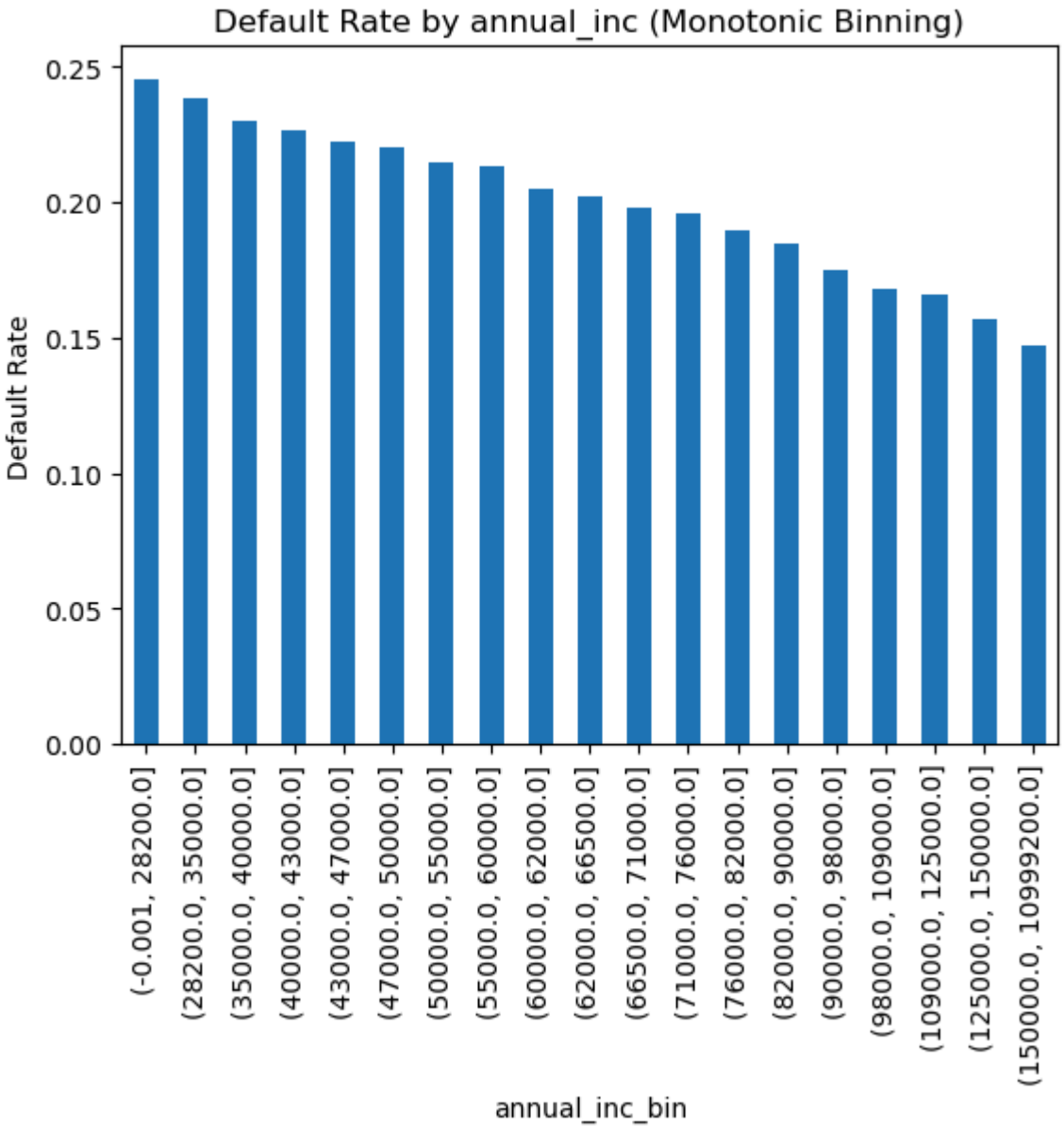

Default Rate by mort_acc (Monotonic Binning)
Default Rate
0.00
0.05
0.10
0.15
0.20
(-0.001, 1.0]
(1.0, 2.0]
(2.0, 3.0]
(3.0, 4.0]
(4.0, 6.0]
(6.0, 51.0]
mort_acc_bin

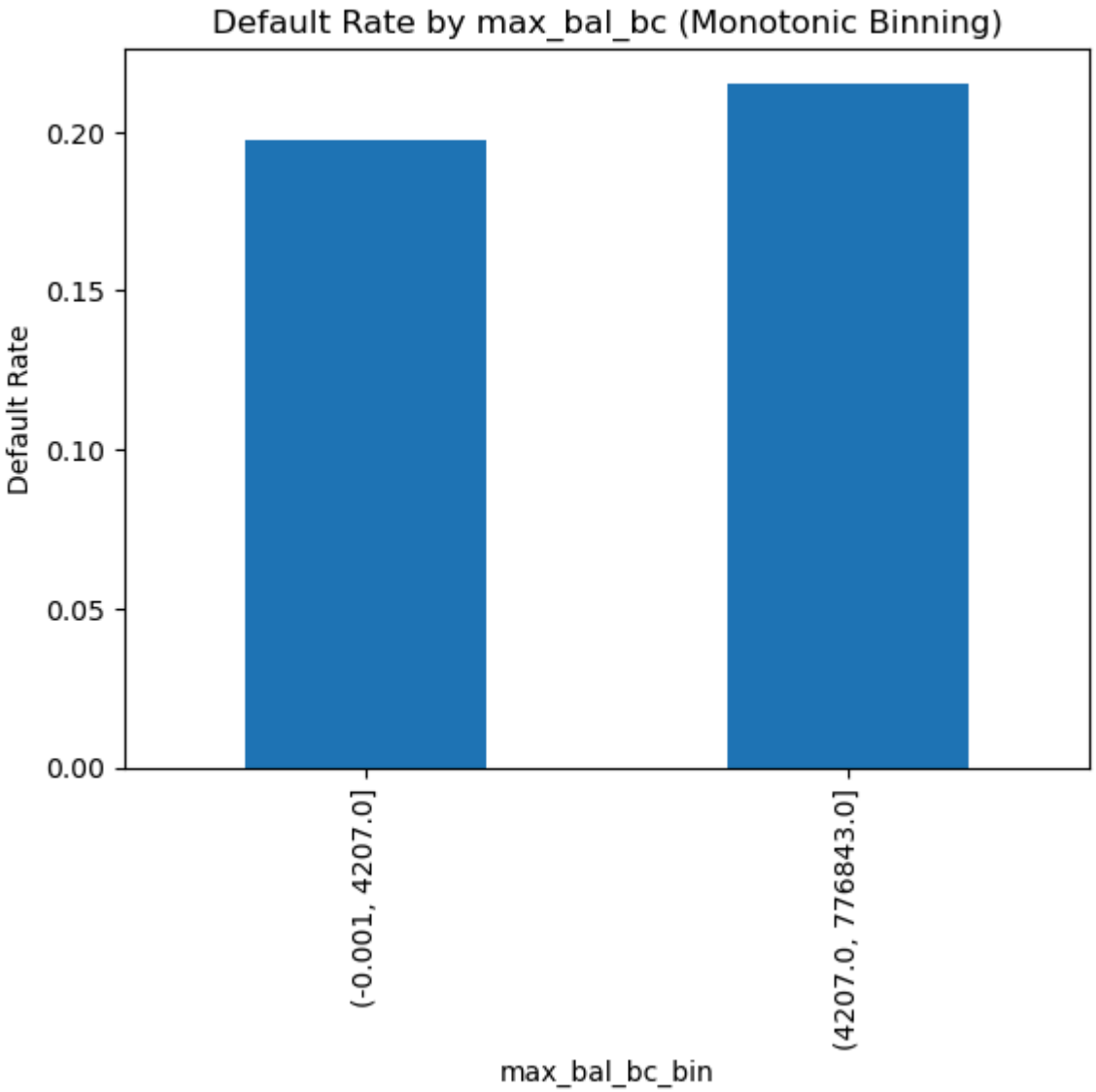

Default Rate by funded_amnt_inv (Monotonic Binning)
Default Rate
0.25
0.20
0.15
0.10
0.05
0.00
(-0.001, 5000.0]
(5000.0, 7000.0]
(7000.0, 9600.0]
(9600.0, 11200.0]
(11200.0, 14000.0]
(14000.0, 16550.0]
(16550.0, 20000.0]
(20000.0, 26125.0]
(26125.0, 40000.0]
funded_amnt_inv_bin

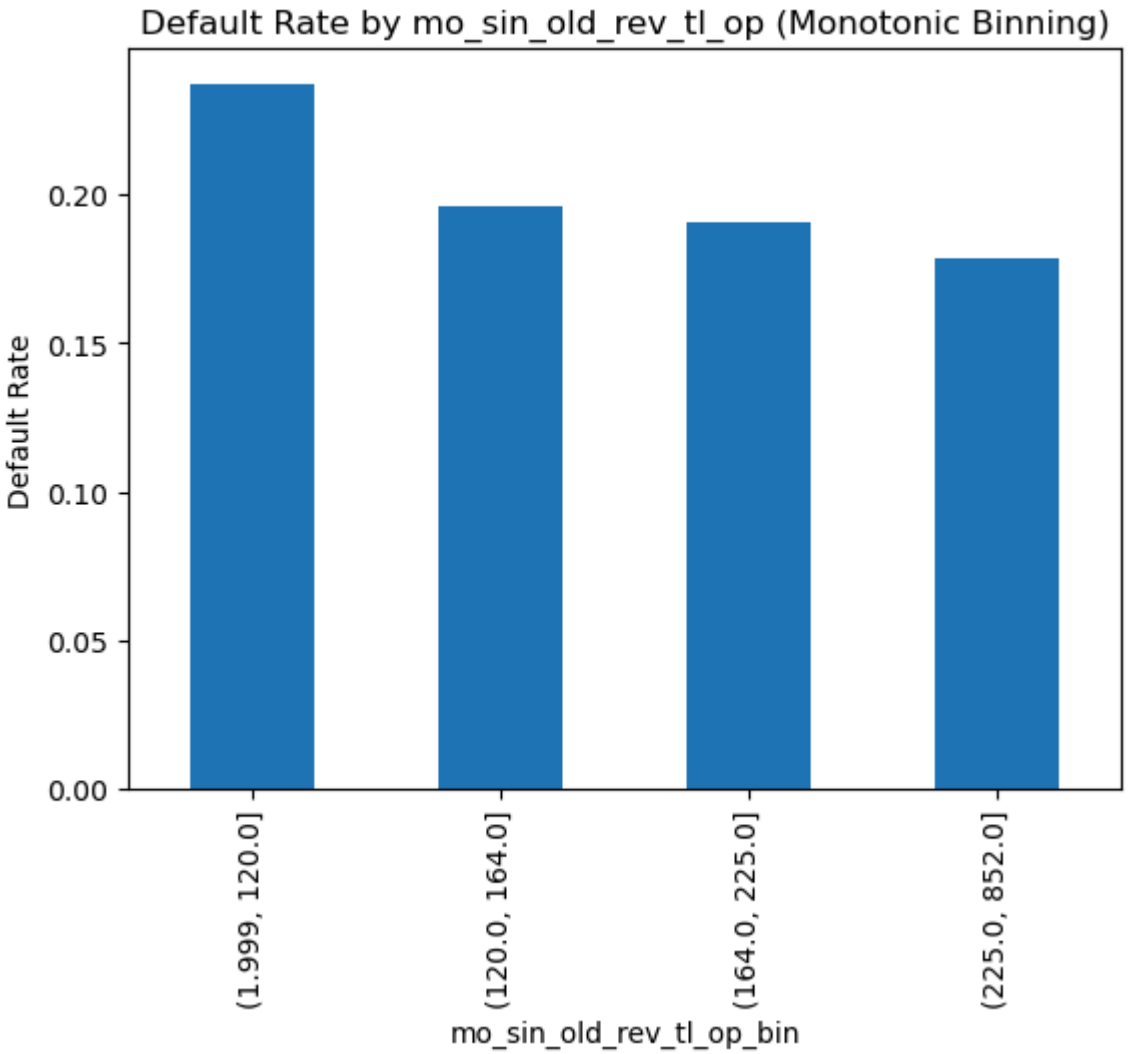

Default Rate by bc_open_to_buy (Monotonic Binning)
Default Rate
0.25
0.20
0.15
0.10
0.05
0.00
(-0.001, 833.0]
(833.0, 2492.0]
(2492.0, 4667.0]
(4667.0, 8261.0]
(8261.0, 17260.0]
(17260.0, 559912.0]
bc_open_to_buy_bin

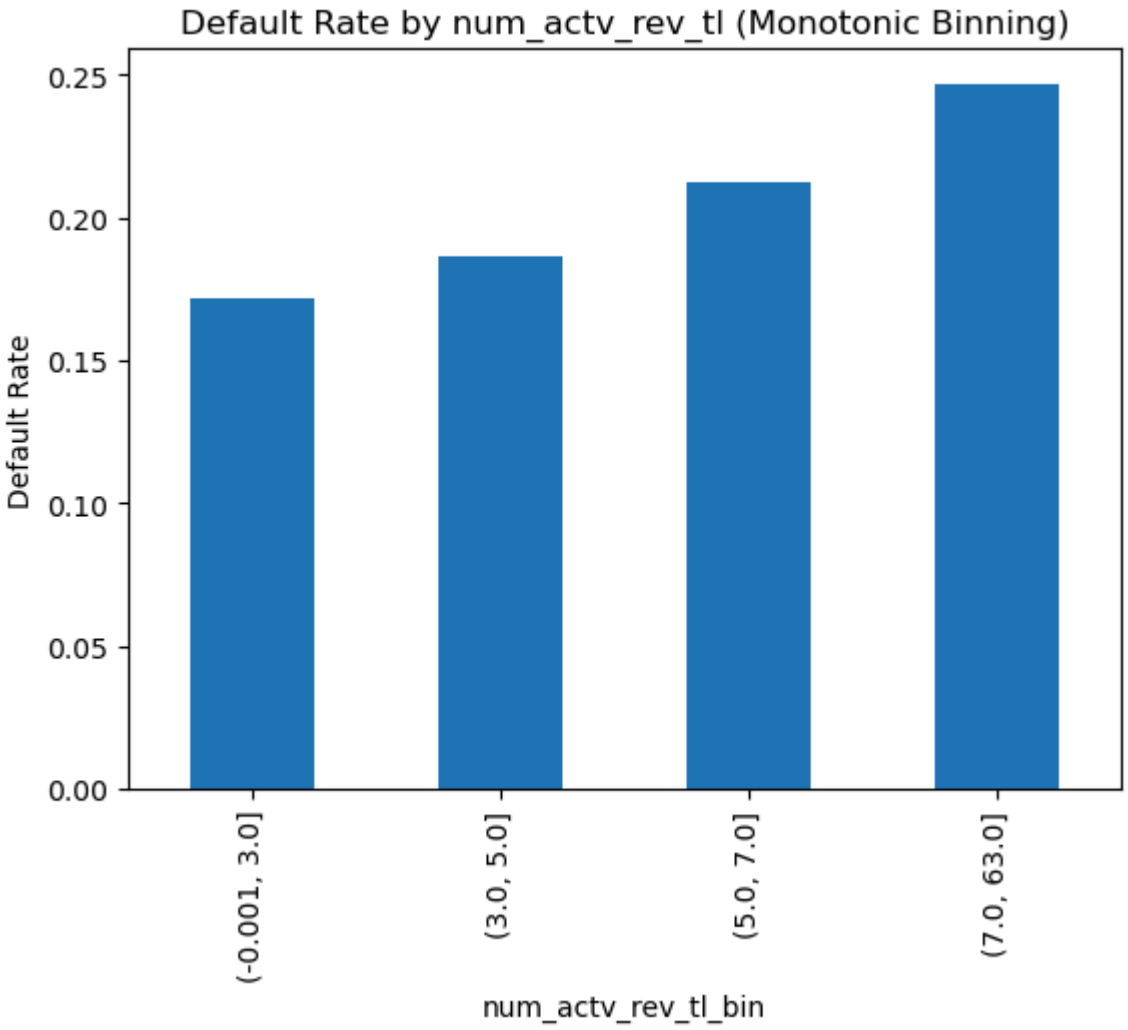

Default Rate by all_util (Monotonic Binning)
Default Rate
0.00
0.05
0.10
0.15
0.20
0.25
(-0.001, 45.0]
(45.0, 60.0]
(60.0, 72.0]
(72.0, 198.0]
all_util_bin

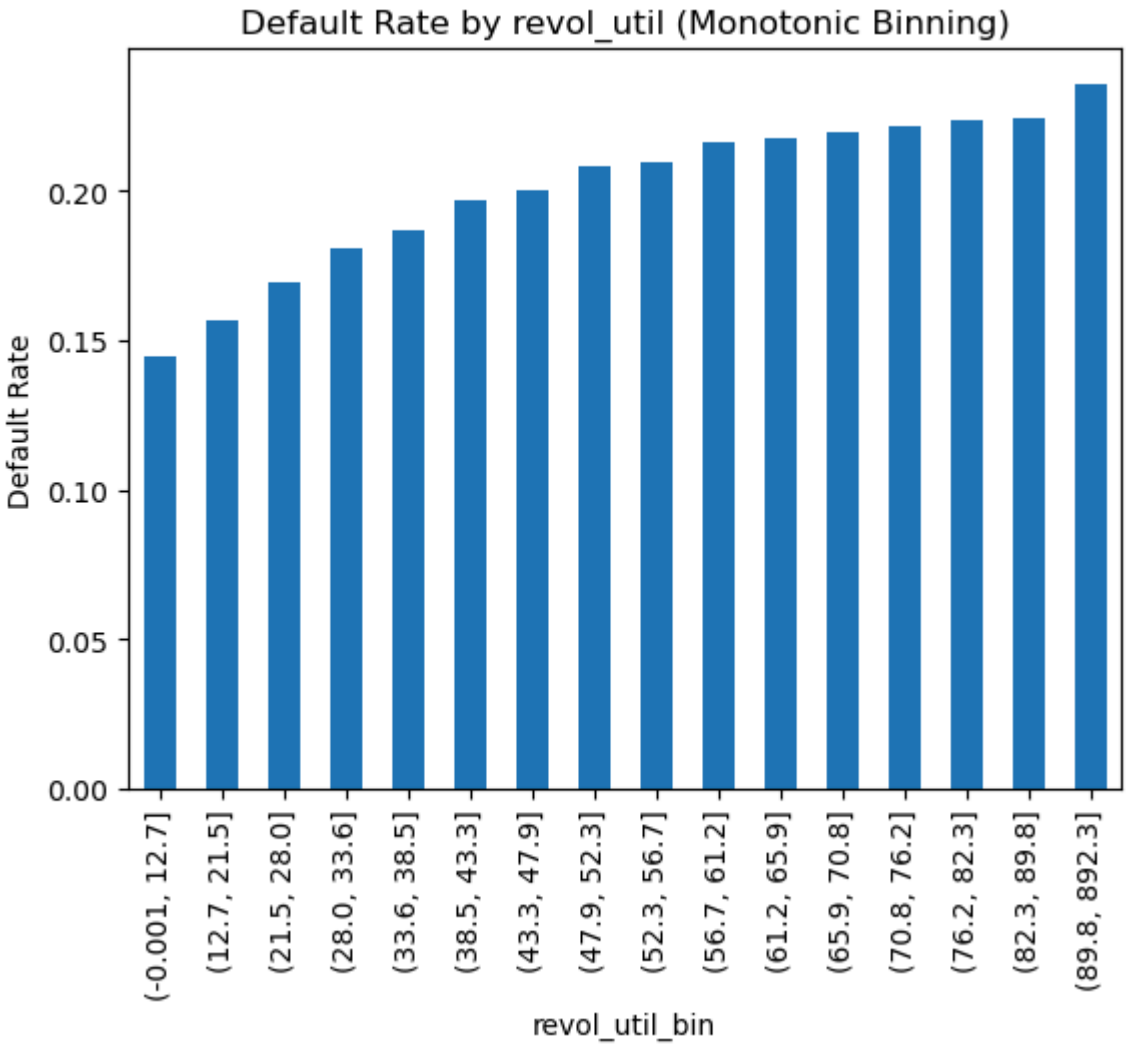

Default Rate by open_rv_24m (Monotonic Binning)
Default Rate
0.00
0.05
0.10
0.15
0.20
0.25
(-0.001, 2.0]
(2.0, 3.0]
(3.0, 53.0]
open_rv_24m_bin


Default Rate by annual_inc (Monotonic Binning)
Default Rate
0.25
0.20
0.15
0.10
0.05
0.00
(-0.001, 28200.0]
(28200.0, 35000.0]
(35000.0, 40000.0]
(40000.0, 43000.0]
(43000.0, 47000.0]
(47000.0, 50000.0]
(50000.0, 55000.0]
(55000.0, 60000.0]
(60000.0, 62000.0]
(62000.0, 66500.0]
(66500.0, 71000.0]
(71000.0, 76000.0]
(76000.0, 82000.0]
(82000.0, 90000.0]
(90000.0, 98000.0]
(98000.0, 109000.0]
(109000.0, 125000.0]
(125000.0, 150000.0]
(150000.0, 10999200.0]
annual_inc_bin


Default Rate by max_bal_bc (Monotonic Binning)
Default Rate
0.20
0.15
0.10
0.05
0.00
(-0.001, 4207.0]
(4207.0, 776843.0]
max_bal_bc_bin


Default Rate by mo_sin_old_rev_tl_op (Monotonic Binning)
Default Rate
0.00
0.05
0.10
0.15
0.20
(1.999, 120.0]
(120.0, 164.0]
(164.0, 225.0]
(225.0, 852.0]
mo_sin_old_rev_tl_op_bin


Default Rate by num_actv_rev_tl (Monotonic Binning)
Default Rate
0.25
0.20
0.15
0.10
0.05
0.00
(-0.001, 3.0]
(3.0, 5.0]
(5.0, 7.0]
(7.0, 63.0]
num_actv_rev_tl_bin



Proceeded With Monotonic Binning Only

IV Calculation

In [19]:
```
target='target'
iv_dict={}
```

```
for col in num_cols:

    table,iv = woe_iv(train_df,col+"_bin",target)

    iv_dict[col]=iv


for col in cat_cols:

    table,iv = woe_iv(train_df,col,target)

    iv_dict[col]=iv

iv_table = pd.DataFrame.from_dict(
iv_dict,orient='index',columns=['IV']
).sort_values(by='IV',ascending=False)

print(iv_table)
```

```
                            IV
debt_settlement_flag  1.258770
sub_grade             0.493861
grade                 0.459630
int_rate              0.455581
term                  0.174030
dti                   0.076034
acc_open_past_24mths  0.062421
verification_status   0.056635
bc_open_to_buy        0.050243
tot_hi_cred_lim       0.037157
all_util              0.034146
funded_amnt_inv       0.033224
mort_acc              0.032077
home_ownership        0.031517
annual_inc            0.030328
open_rv_24m           0.029516
num_actv_rev_tl       0.028881
revol_util            0.026578
mo_sin_old_rev_tl_op  0.018285
emp_length            0.011444
max_bal_bc            0.001948
```

Select Variables

In [20]:
```
selected = iv_table[iv_table['IV']>=0.03].index.tolist()
```

WOE Transformation

In [21]:
```
for col in selected:

    if col in num_cols:

        table,iv = woe_iv(train_df,col+"_bin",target)

        woe_map = table['WOE'].to_dict()

        train_df[col+"_woe"] = train_df[col+"_bin"].map(woe_map)
```

```
        val_df[col+"_woe"] = val_df[col+"_bin"].map(woe_map)

    else:

        table,iv = woe_iv(train_df,col,target)

        woe_map = table['WOE'].to_dict()

        train_df[col+"_woe"] = train_df[col].map(woe_map)
        val_df[col+"_woe"] = val_df[col].map(woe_map)


woe_cols = [c for c in train_df.columns if '_woe' in c]
```

In [22]:
```
woe_cols
```

Out[22]:
```
['debt_settlement_flag_woe',
 'sub_grade_woe',
 'grade_woe',
 'int_rate_woe',
 'term_woe',
 'dti_woe',
 'acc_open_past_24mths_woe',
 'verification_status_woe',
 'bc_open_to_buy_woe',
 'tot_hi_cred_lim_woe',
 'all_util_woe',
 'funded_amnt_inv_woe',
 'mort_acc_woe',
 'home_ownership_woe',
 'annual_inc_woe']
```

Dropping Statistically insignificant variable(grade_woe) after first Logistic model run

In [23]:
```
woe_cols.remove("grade_woe")
```

Logistic Regression PD Model

In [24]:
```
X_train = train_df[woe_cols]
y_train = train_df['target']

X_train = sm.add_constant(X_train)

logit = sm.Logit(y_train,X_train)

result = logit.fit()

print(result.summary())
```

```
Optimization terminated successfully.
         Current function value: 0.417270
         Iterations 13
                           Logit Regression Results
==============================================================================
Dep. Variable:                 target   No. Observations:               912546
Model:                          Logit   Df Residuals:                   912531
Method:                           MLE   Df Model:                           14
Date:                Fri, 13 Mar 2026   Pseudo R-squ.:                  0.1678
Time:                        22:07:29   Log-Likelihood:            -3.8078e+05
converged:                       True   LL-Null:                   -4.5757e+05
Covariance Type:            nonrobust   LLR p-value:                     0.000
==============================================================================================
                                coef    std err          z      P>|z|      [0.025      0.975]
----------------------------------------------------------------------------------------------
const                        -1.3827      0.008   -174.888      0.000      -1.398      -1.367
debt_settlement_flag_woe     -1.0183      0.056    -18.126      0.000      -1.128      -0.908
sub_grade_woe                -0.7781      0.018    -42.076      0.000      -0.814      -0.742
int_rate_woe                  0.0847      0.019      4.489      0.000       0.048       0.122
term_woe                     -0.4825      0.008    -58.759      0.000      -0.499      -0.466
dti_woe                      -0.3593      0.011    -31.679      0.000      -0.382      -0.337
acc_open_past_24mths_woe     -0.6802      0.012    -55.147      0.000      -0.704      -0.656
verification_status_woe      -0.2077      0.013    -16.054      0.000      -0.233      -0.182
bc_open_to_buy_woe           -0.2296      0.015    -15.158      0.000      -0.259      -0.200
tot_hi_cred_lim_woe          -0.5187      0.023    -22.586      0.000      -0.564      -0.474
all_util_woe                 -0.5422      0.016    -34.184      0.000      -0.573      -0.511
funded_amnt_inv_woe          -0.7630      0.021    -36.838      0.000      -0.804      -0.722
mort_acc_woe                 -0.4636      0.020    -23.048      0.000      -0.503      -0.424
home_ownership_woe           -0.5421      0.021    -26.425      0.000      -0.582      -0.502
annual_inc_woe               -0.6765      0.023    -29.925      0.000      -0.721      -0.632
==============================================================================================
```

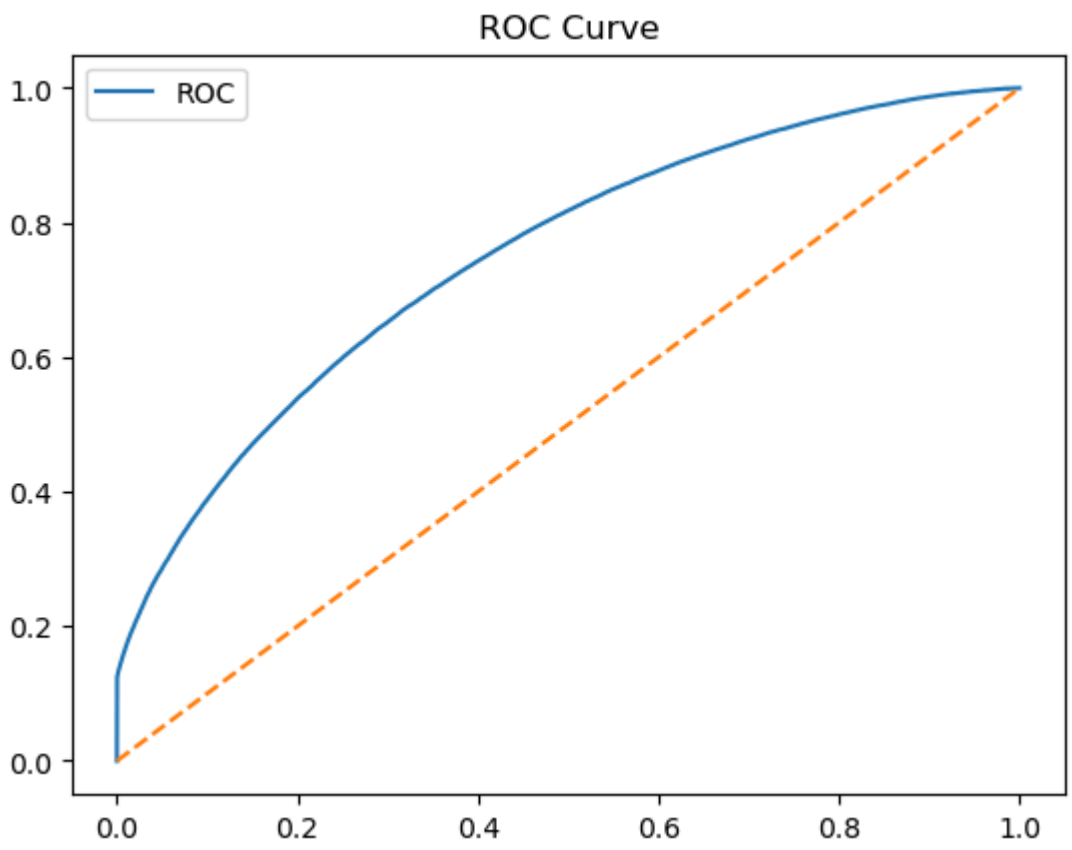
Validation Scoring

In [25]:
```
X_val_model = val_df[woe_cols]

X_val_model = sm.add_constant(X_val_model)

val_df['PD'] = result.predict(X_val_model)
```

Model Performance

In [26]:
```
auc = roc_auc_score(val_df['target'],val_df['PD'])

gini = 2*auc-1

print("Validation AUC:",auc)
print("Validation Gini:",gini)

fpr,tpr,_ = roc_curve(val_df['target'],val_df['PD'])

ks = max(tpr-fpr)

print("KS:",ks)

plt.plot(fpr,tpr,label="ROC")
plt.plot([0,1],[0,1],'--')
plt.legend()
plt.title("ROC Curve")
plt.show()
```

```
Validation AUC: 0.7488156711391334
Validation Gini: 0.4976313422782668
KS: 0.35310073489368987
```



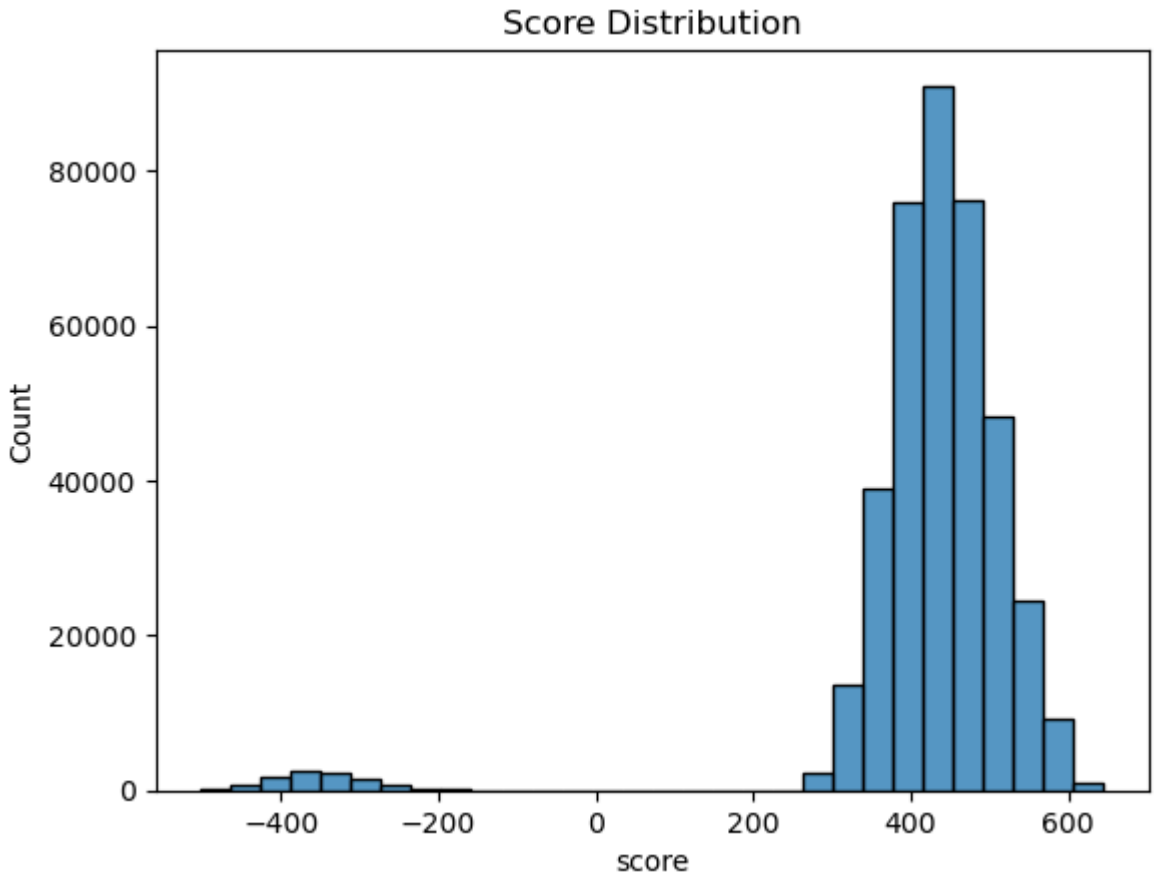
Score Scaling

In [27]:
```
PDO = 50

factor = PDO/np.log(2)

base_score = 600

base_odds = 50

offset = base_score - factor*np.log(base_odds)

val_df['score'] = offset - factor*np.log(
```

```
val_df['PD']/(1-val_df['PD'])

)
```

Score Distribution

In [28]:
```
sns.histplot(val_df['score'],bins=30)

plt.title("Score Distribution")

plt.show()
```

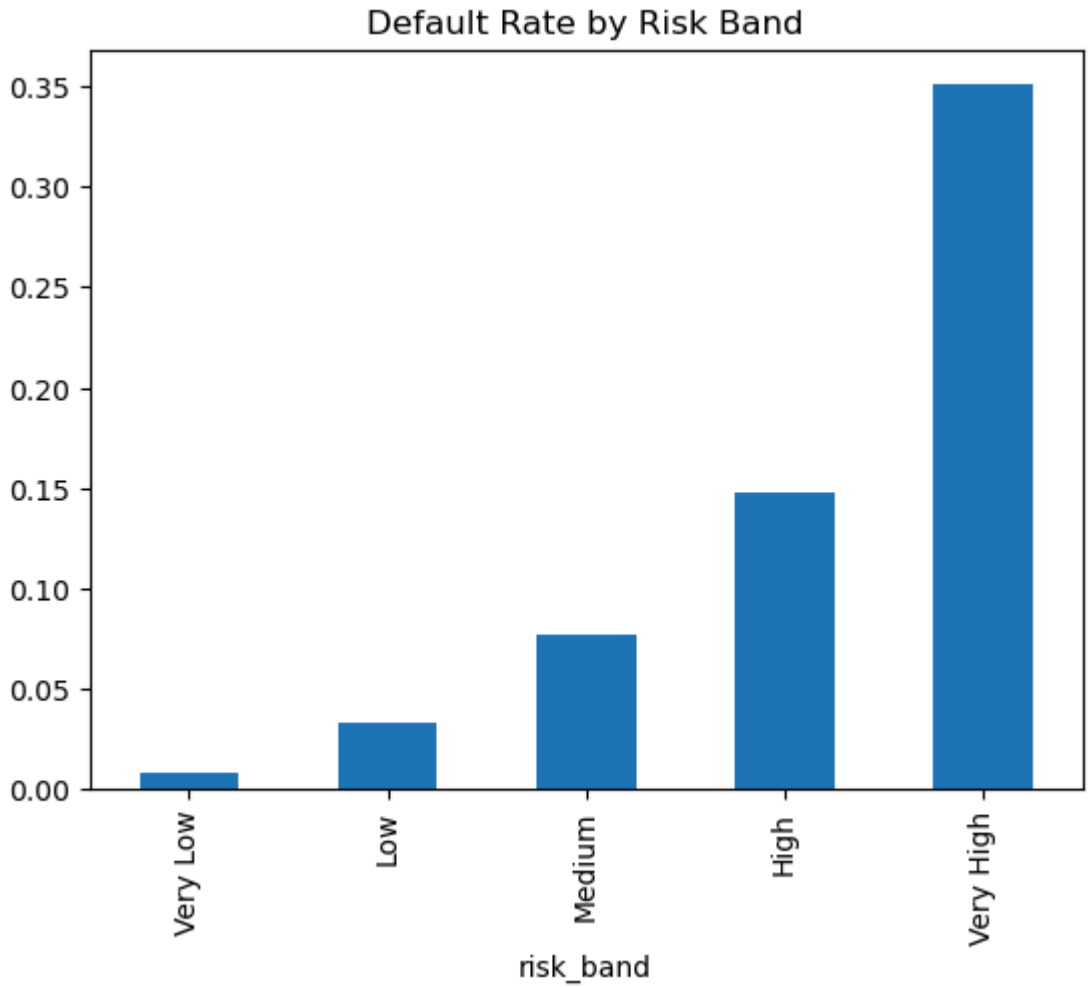


Risk Bands

In [29]:
```
val_df['risk_band'] = pd.cut(
val_df['PD'],
bins=[0,0.02,0.05,0.1,0.2,1],
labels=["Very Low","Low","Medium","High","Very High"]
)
```

```python
risk_summary = val_df.groupby('risk_band')['target'].mean()

print(risk_summary)

risk_summary.plot(kind='bar')

plt.title("Default Rate by Risk Band")

plt.show()
```

```
risk_band
Very Low     0.008371
Low          0.033400
Medium       0.077229
High         0.148193
Very High    0.350427
Name: target, dtype: float64
```



Population Stability Index (PSI)

In [30]:
```
# Predict PD on Development Sample

X_dev_model = train_df[woe_cols]
X_dev_model = sm.add_constant(X_dev_model)

train_df['PD'] = result.predict(X_dev_model)
```

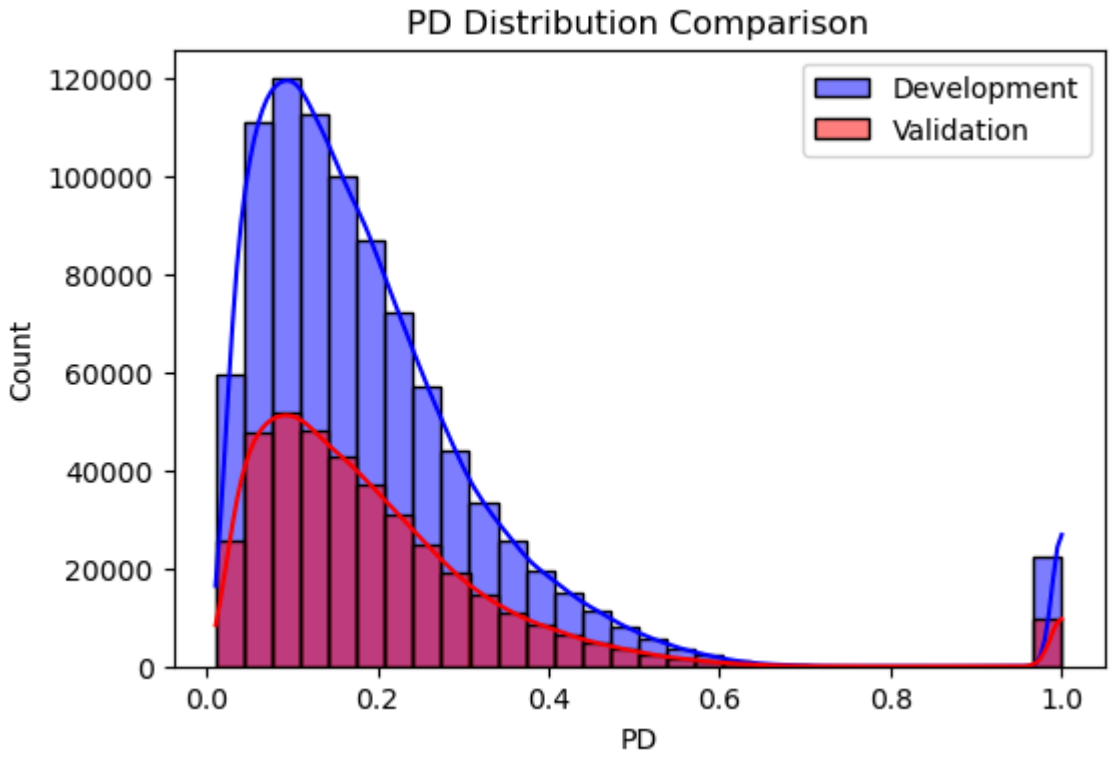
```python
# PSI Function

def calculate_psi(expected, actual, bins=10):

    expected_counts, bin_edges = np.histogram(expected, bins=bins)
    actual_counts, _ = np.histogram(actual, bins=bin_edges)

    expected_perc = expected_counts / len(expected)
    actual_perc = actual_counts / len(actual)

    psi_values = (actual_perc - expected_perc) * np.log(
        (actual_perc + 1e-6) / (expected_perc + 1e-6)
    )

    psi_table = pd.DataFrame({
        "Bin": range(1, bins+1),
        "Dev %": expected_perc,
        "Val %": actual_perc,
        "PSI": psi_values
    })

    # Add Total PSI row
    total_psi = psi_table["PSI"].sum()

    total_row = pd.DataFrame({
        "Bin": ["Total"],
        "Dev %": [psi_table["Dev %"].sum()],
        "Val %": [psi_table["Val %"].sum()],
        "PSI": [total_psi]
    })

    psi_table = pd.concat([psi_table, total_row], ignore_index=True)

    return total_psi, psi_table


# Calculate PSI

psi_value, psi_table = calculate_psi(
```

```
    train_df['PD'],
    val_df['PD'],
    bins=10
)

print("Population Stability Index (PSI):", psi_value)

print("\nPSI Table:")
print(psi_table)
```

```
Population Stability Index (PSI): 5.653073424123176e-05

PSI Table:
      Bin     Dev %     Val %           PSI
0       1  0.318198  0.318940  1.731231e-06
1       2  0.328165  0.326304  1.058448e-05
2       3  0.189982  0.190656  2.388692e-06
3       4  0.086425  0.087110  5.403163e-06
4       5  0.038105  0.037753  3.270560e-06
5       6  0.012581  0.012654  4.226310e-07
6       7  0.001913  0.002076  1.330520e-05
7       8  0.000042  0.000074  1.842224e-05
8       9  0.000000  0.000000  0.000000e+00
9      10  0.024588  0.024432  1.002537e-06
10  Total  1.000000  1.000000  5.653073e-05
```

PSI Visualization

In [31]:
```
plt.figure(figsize=(6,4))

sns.histplot(train_df['PD'], color='blue', label='Development', bins=30, kde=True)

sns.histplot(val_df['PD'], color='red', label='Validation', bins=30, kde=True)

plt.title("PD Distribution Comparison")

plt.legend()

plt.show()
```


PD Distribution Comparison
Development
Validation
Count
PD
0
20000
40000
60000
80000
100000
120000
0.0
0.2
0.4
0.6
0.8
1.0